PhyloPOMP.jl

A Technical Report in KLI Language

Generated from Milestones M00–M13 Review

Reference: King, Lin, & Ionides, "Exact Phylodynamic Likelihood via Structured Markov Genealogy Processes"

August 21, 2026

Abstract

This document describes the PhyloPOMP.jl Julia package using the mathematical vocabulary of the King–Lin–Ionides (KLI) framework for exact phylodynamic likelihood via structured Markov genealogy processes. We describe: (i) the purpose of the repository, (ii) how each component functions—with architecture diagrams—in terms of KLI's population process, production, saturation, binomial ratio, and filter equation concepts, (iii) the mathematical derivations that ground the implementation, and (iv) why the verification strategy escapes circularity, anchored by a Kingman-coalescent/Moran-model external check and, separately, by a falsification-tested generalization check against SI2R, an epidemic model specification never consulted during this compiler's development.

Contents

1 Purpose: Exact Phylodynamic Likelihood for Structured Models

1.1 The phylodynamic likelihood problem

Let $\mathbf{X}_t$ be a time-inhomogeneous Markov jump process on a state space $\mathcal{X}$ with transition rates $\alpha(t, x, x')$ and initial density p_0. In epidemiology, $\mathbf{X}_t$ represents the state of a disease transmission system—numbers of susceptible, exposed, infectious, and recovered hosts—and its jumps represent births, deaths, migrations, samplings, and neutral demographic events.

The *phylodynamic likelihood* is the probability density of an observed genealogy $\mathbf{P}$ (a pruned, sample-only phylogenetic tree) given the dynamic model:

$$\mathcal{L} = f(\mathbf{P} \mid D) = \int w(T, x)\, \mathrm{d}x, \tag{1}$$

where $w(t, x)$ satisfies a *filter equation* whose structure is completely determined by the population model D.

1.2 What PhyloPOMP.jl does

PhyloPOMP.jl is a Julia implementation of the KLI theory. Its purpose is to provide:

(a) A **model DSL** (`@mgp`, `@event`) for declaring arbitrary discretely-structured Markov population models—specifying compartments, demes $\mathcal{D}$, event marks $\mathcal{U}$, hazard functions α_u, state-change vectors Δ_u, and production vectors r^u—without restricting the user to coalescent or linear-birth-death special cases.

(b) A **generic compiler pipeline** that, given any such model, mechanically derives the quantities needed for KLI's filter equation: the binomial ratio φ_u, the reduced compatibility factor Φ_u, the decay λ, and the driver/boost decomposition.

(c) A **forward simulator** of genealogy processes $\mathbf{G}_t$ and a **sequential Monte Carlo (SMC) filter** that numerically solves the filter equation to compute (1).

The package is the Julia companion to the R package phylopomp. The compiler pipeline (items (a)–(b)) was built over milestones M00–M13 and validated against hand-coded reference implementations and, crucially, against the textbook Kingman coalescent—an external, non-project-authored source of truth.

1.3 Terminology

Sections 2–4 mix vocabulary from three quite different places, and the reader is owed an explicit statement of which is which. Some words are **KLI's**: they appear in the reference paper and carry its technical meaning. Some are **older and wider than both KLI and this package**: standard compiler-theory, combinatorics, point-process, or population-genetics terms that KLI and PhyloPOMP.jl merely adopt. And some are **this codebase's own coinages**: internal names for internal objects, invented here, with no standing outside this repository. Blurring the three would make the implementation look more theory-sanctioned than it is, so they are kept apart below. Definitions here are deliberately terse; the full development is in §3.

1.3.1 Index

Term	Provenance	Full treatment
DSL	general CS	§1.3 only
IR	general CS	§1.3 only
Chu–Vandermonde identity	classical combinatorics	§2, Eq. (5)
hazard α_u	point-process/survival, used by KLI	§3
deme $\mathcal{D}$	population genetics, used by KLI	§3
jump mark $u \in \mathcal{U}$	KLI	§3
MGP	KLI (paper title)	§3
production r^u	KLI	§3
saturation s	KLI	§2, Eq. (2)
lineage count ℓ	KLI	§3
coloring $Y = (d, m)$	KLI	§3
compatibility indicator Q_u	KLI	§3, Thm. 1
binomial ratio φ_u	KLI (Eq. 9)	§2, Eq. (3)
reduced factor Φ_u	KLI	§2, Eq. (4)
regular / singular event	KLI	§3
inline node	KLI	§3
σ / χ / κ	KLI	§3
driver β_u	KLI	§2, Eq. (7)
boost B_u	KLI	§2, Eq. (7)
decay λ	KLI	§2, Eq. (6)
proposal kernel π_u	KLI	§2, Eq. (7)
Population IR / Filter IR	this project	§2
RegularFlow / SingularFlow / OutflowImbalanceTerm	this project	§2
Identity / InlineSameDeme / CrossDeme / Fork	this project	§2
`:noop` / `:cross` / `:fork`	this project	§2
Gate 5	this project	§4
M00–M13	this project	§1.3 only

1.3.2 Vocabulary that predates and is independent of both KLI and this package

These words were not coined by KLI and are not specific to phylodynamics. They are used here in their ordinary, pre-existing sense.

DSL (domain-specific language). A general term from compiler theory and programming-language design: a small language purpose-built for one narrow domain, as opposed to a general-purpose language in which any program can be written. In PhyloPOMP.jl the DSL is the pair of Julia macros `@mgp` and `@event`, in which a user declares a compartmental epidemic/phylodynamic model—its compartments, its demes $\mathcal{D}$, and one line per jump mark giving that mark's rate, its state change, and its genealogical move. Nothing about the phrase "DSL" is KLI's or this

project's; it names a well-established implementation strategy that happens to be the one used here.

IR (intermediate representation). Also a general compiler-theory term: the data structure a compiler hands from one stage to the next in place of raw source text, so that later stages analyse a normalised object rather than re-parsing. A compiler typically has several, at descending levels of abstraction. PhyloPOMP.jl has four. The *Population IR* is the `MGPModel` struct and its vector of `Event` structs, produced directly by the macros. The *full KLI transition set* and then the *reduced KLI transition set* are produced by the φ_u/classification and m-reduction stages. The *Filter IR* is the final, bucketed form consumed by filter assembly. Again: "IR" is borrowed vocabulary; the specific *names* of those four IRs are this project's, and are listed as such below.

Chu–Vandermonde identity. A classical combinatorial identity, known long before KLI and entirely independent of it: $\sum_k \binom{m}{k}\binom{n}{p-k} = \binom{m+n}{p}$. KLI *applies* it—it is what makes the φ_u over a mark's saturations a partition of unity, Eq. (5)—but did not invent it, and neither did this project. In verification it is therefore a check of the compiler's internal arithmetic against a known theorem, not a check against a model-specific external reference.

hazard, $\alpha_u(t, x, x')$. Standard point-process and survival vocabulary: the instantaneous rate of an event, so that the event occurs in $[t, t + \mathrm{d}t)$ with probability $\alpha_u\,\mathrm{d}t + o(\mathrm{d}t)$. KLI uses the word in exactly this ordinary sense, indexed by jump mark u. In the DSL a hazard is whatever expression appears after `rate=`.

deme. Standard population-genetics vocabulary for a sub-population within which individuals are exchangeable and between which individuals may move. KLI adopts the word unchanged; a structured model's deme set is written $\mathcal{D}$. In an epidemic model the demes are the compartments a tracked lineage can actually occupy—for SEIR, $\mathcal{D} = \{\mathsf{E}, \mathsf{I}\}$; susceptible and recovered hosts carry no lineage and so are compartments but not demes.

1.3.3 Genuine KLI vocabulary

These terms are the reference paper's, with the paper's meanings. They are stated briefly here only so that Sections 2 and 4 are readable; §3 gives the definitions in full and should be taken as authoritative.

MGP (Markov genealogy process). KLI's central object and the source of the `@mgp` macro's name: the joint process $(\mathbf{X}_t, \mathbf{G}_t)$ of a Markov population process and the genealogy it generates, together with the mark process $\mathbf{U}_t$ recording which kind of event occurred.

jump mark $u \in \mathcal{U}$. KLI's label for one *kind* of event in the population model—an infection, a progression, a recovery, a sampling. Each mark carries its own hazard α_u, state-change vector Δ_u, and production vector r^u. One `@event` line in the DSL declares exactly one jump mark.

production, $r^u \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$. KLI's count, per deme, of the *emergent* individuals a mark produces—the "slots" that must be filled at the event. A same-deme birth has $r^u = (2, 0)$: parent and offspring both emerge in the first deme. A migration has a single slot in the destination deme. A death or a destructive sample has $r^u = 0$.

saturation, $s \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$**.** KLI's count, per deme, of how many of that deme's r^u production slots are filled by *tracked* lineages—lineages ancestral to a sample and therefore present in the pruned genealogy. Necessarily $0 \leq s_i \leq \min(r_i^u, \ell_i)$; see Eq. (2).

lineage count, $\ell \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$**.** KLI's count, per deme, of currently tracked lineages. Together with the population occupancy n it determines every combinatorial quantity in the filter: ℓ_i of the n_i individuals in deme i are tracked and $n_i - \ell_i$ are not.

coloring, $Y = (d, m)$**.** KLI's decoration of the pruned genealogy: each branch carries a deme d (which sub-population the lineage is in) and an event indicator m (whether that branch was involved in the event just processed). The filter equation acts on the *reduced*, deme-only state, and the elimination of m is a distinct compiler stage.

compatibility indicator, $Q_u \in \{0, 1\}$**.** KLI's zero/one factor in Theorem 1 stating whether a proposed coloring transition $\tilde{Y} \to Y$ is consistent with mark u at all. Incompatible transitions get $\varphi_u = 0$.

binomial ratio, φ_u**.** KLI Eq. 9, the time-local factor of Theorem 1: the probability that a mark-u event fills its slots in exactly the tracked/untracked pattern the genealogy requires, under uniform exchangeable selection within each deme. See Eq. (3).

reduced compatibility factor, Φ_u**.** The aggregate of the φ_u over all saturations that reduce to the same deme-only outcome, Eq. (4). Distinct from Q_u: Q_u is a zero/one admissibility flag inside a single φ_u, whereas Φ_u is a sum of φ_u values.

regular / singular event. KLI's split of the filter equation. A time is *singular* when it coincides with an observed event of the pruned genealogy (a sampling, a branch point) and *regular* otherwise. The two cases obey structurally different equations—a differential equation between observations, a kernel update at them. The DSL's `kind=regular` / `kind=singular` annotation records which.

inline node. KLI's term for a node of the genealogy at which a tracked lineage is involved in an event but no branching occurs—the lineage passes through, possibly changing deme. Migrations and single-slot-filled births produce inline nodes (KLI §§3.1, 5.1).

σ, χ, κ (swap, chop, fork). KLI's operators on a coloring (KLI §5.1): σ moves a lineage from one deme to another, χ removes a lineage, κ splits one lineage into two at a branch point. The DSL's `move=swap(...)`, `move=chop(...)`, `move=fork(...)` are direct spellings of these three.

driver, boost, decay. KLI's three components of the filter equation. The *driver* $\beta_u = \alpha_u \pi_u$ is the rate at which the particle filter actually proposes a mark-u move; the *boost* $B_u = \Phi_u / \pi_u$ is the importance-sampling correction that reweights that proposal onto the target; the *decay* λ is the exponential attenuation of particle weight between events, arising from sampling hazard and sub-threshold removal (Eqs. (6)–(7)).

proposal kernel, π_u**.** The probability with which the SMC algorithm's proposal chooses the reduced outcome associated with mark u. It is a free implementation choice, not a property of the model; the boost $B_u = \Phi_u / \pi_u$ exists precisely to cancel that freedom, so that the likelihood estimate does not depend on it.

1.3.4 This project's own naming

The following names appear nowhere in the KLI paper and nowhere in general computer science. They are internal labels this codebase gave to its own objects. Several of them denote genuinely KLI concepts—but the *names* are ours, and a reader should not take their appearance as evidence that the paper endorses the particular decomposition they encode.

Population IR, Filter IR. This project's names for two of its four intermediate representations (see "IR" above): respectively, the `MGPModel`/`Event` structs emitted by the macros, and the bucketed classification of reduced transitions consumed by filter assembly.

RegularFlow, SingularFlow, OutflowImbalanceTerm. This project's three Filter-IR bucket names. A reduced transition is a `RegularFlow` if it contributes $\alpha_u \Phi_u$ to the inflow between observations, a `SingularFlow` if it fires at an observed event time, and an `OutflowImbalanceTerm` if it is fork mass that the filter resolves through the asymmetry between full-hazard outflow and non-fork inflow rather than through λ. The underlying regular/singular split is KLI's; the three bucket names are not.

Identity, InlineSameDeme, CrossDeme, Fork. This project's four-way classification of saturations, assigned by `full_transitions` according to sum(s) and the occupied slot's deme: no slot filled ($s = \mathbf{0}$), one slot filled in the ancestral deme, one slot filled in a different deme, and two or more slots filled. The concepts behind them are KLI's—the middle two are inline nodes, realised by σ; the last is κ—but these four class names are this codebase's coinage.

`:noop`, `:cross`, `:fork`. This project's three reduction-group tags, emitted by `reduce_event_indicator` when it eliminates the event indicator m. Identity and InlineSameDeme both collapse into the single `:noop` group (they are indistinguishable once m is gone); CrossDeme and Fork keep their own groups. As with the class names, the tags are ours.

Gate 5. This project's name for one specific validation check: run a compiled filter and an independently hand-coded reference filter for the same model on identical seeds with $N_p = 1$, and require their log-likelihoods to agree to floating-point noise. It is a numerical *equivalence* check between two implementations, not a check against any external source of truth. "Gate 5" is purely internal shorthand; it has no meaning in KLI or in the wider literature.

M00–M13. This project's milestone tags, used throughout as a shorthand for *when*, and therefore *against which models*, a given piece of compiler code was written. They matter to the non-circularity argument of §4: code written at M02–M04 predates every model it is later exercised on, and could not have been fitted to those models' answers.

Remark 1 (Reading the provenance labels)**.** Where later sections say a quantity "matches KLI Eq. 9" they are claiming agreement with the paper. Where they say a transition is "classified `CrossDeme`" they are describing this implementation's bookkeeping and claiming nothing about the paper. The distinction is load-bearing in §4, where the strength of each verification layer depends entirely on how far outside this project its reference lies.

2 Architecture: How Each Part Functions

2.1 The compiler pipeline

The central contribution of the M00–M13 milestone sequence is a *compiler*: a chain of composable, independently-tested Julia functions that lowers a declarative model specification into executable

filter code. Each stage corresponds to a specific mathematical object in the KLI framework.

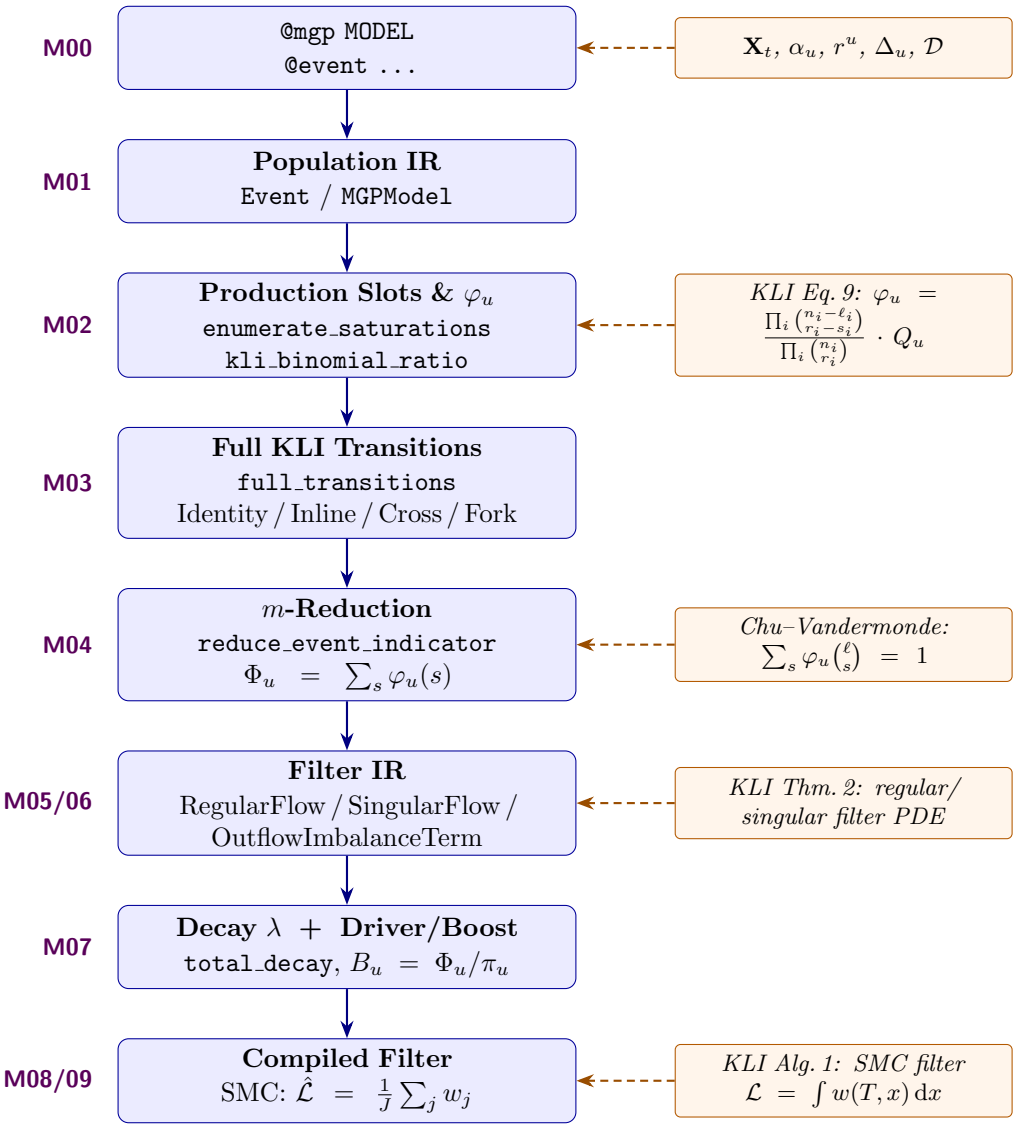


Figure 1: The PhyloPOMP.jl compiler pipeline. Each stage (blue boxes, left) corresponds to a milestone and implements a specific KLI mathematical object (orange boxes, right). Arrows show data flow; dashed arrows show the KLI equation each stage realizes.

The stages are not merely conceptually separate: each is a distinct source file exporting ordinary callable functions, and each consumes only the previous stage's output type. Stage 2 reads nothing but the production vector r^u stored in the Population IR; Stage 3 consumes Stage 2's saturations and φ_u values without re-deriving either; Stage 4 consumes Stage 3's transition vector; Stage 5 consumes Stage 4's reduced transitions. Because the interfaces are values rather than call-backs, every stage is separately testable against a hand-derived oracle, which is what makes the verification argument of Section 4 possible at all.

Reading the code excerpts. Each stage below carries a literal excerpt of the Julia source that implements it, cited by file and line number. One systematic substitution is applied throughout:

the sources use Unicode identifiers—φ, ℓ, Φ, α, θ, Δ, π, and Greek parameter names such as β, σ, γ, ω, ψ—which are transliterated to their ASCII spellings (`phi`, `ell`, `Phi`, `alpha`, `theta`, `Delta`, `pi`, `beta`, `sigma`, ...) so that they typeset inside `verbatim`. A line consisting only of `[...]` marks elided guard or comment code; the elided range is always stated in the surrounding prose. Everything else is the file's own text at the cited lines.

2.2 Stage 1: Model Declaration and Population IR (M00–M01)

The user declares a population model using Julia macros. The shipped SEIR declaration is `mgp_macro.jl:188--198`:

```
@mgp SEIR begin
    compartments = (S, E, I, R)
    demes = (E, I)
    params = (beta, sigma, gamma, omega, psi, chi, N)

    @event infection   rate=beta*S*I/N pop=(S=-1, E=+1) move=fork(I => E, I) kind=regular
    @event progression rate=sigma*E     pop=(E=-1, I=+1) move=swap(E => I)    kind=regular
    @event recovery    rate=gamma*I     pop=(I=-1, R=+1) move=chop(I)         kind=regular
    @event waning      rate=omega*R     pop=(R=-1, S=+1) move=none            kind=regular
    @event sampling    rate=psi*I     pop=()           move=sample(I)      kind=singular
end
```

In KLI terms, this declares:

- The **deme set** $\mathcal{D} = \{\mathsf{E}, \mathsf{I}\}$, the compartments (S, E, I, R), and the parameter names. The macro requires all three declarations—`mgp_macro.jl:169--171` errors if any of `compartments`, `demes`, or `params` is absent—and `mgp_macro.jl:176` additionally requires $\mathcal{D}$ to be a *subset* of the compartments, enforcing KLI's convention that a deme is a lineage-carrying compartment rather than a separate object.
- The **jump marks** $\mathcal{U} = \{\text{infection}, \text{progression}, \text{recovery}, \text{waning}, \text{sampling}\}$.
- For each mark $u \in \mathcal{U}$: the **hazard function** $\alpha_u(t, x, x')$ (from `rate=`), the **state-change vector** Δ_u (from `pop=`), the **production vector** $r^u \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$ (inferred from `move=`), the **event type** (Birth/Death/Migration/Sample/Neutral), and whether the event is **regular** ($t \notin \text{event}(\mathbf{P})$) or **singular** ($t \in \text{event}(\mathbf{P})$).

The macro's only semantic act is a symbol rewrite: `_rewrite` (`mgp_macro.jl:9--18`) walks the `rate=` expression tree and replaces a bare compartment symbol by a field access on the state tuple and a bare parameter symbol by a field access on the parameter tuple, so `rate=beta*S*I/N` becomes the closure `(x,theta) -> theta.beta*x.S*x.I/theta.N`. No likelihood quantity is generated at macro-expansion time; the macro emits data only.

Each declaration lowers into one `Event` struct, and the vector of them into an `MGPModel`—the *Population IR*. These two structs are the entire interface between the modelling language and every downstream stage (`mgp.jl:59--69` and `mgp.jl:77--82`):

```
struct Event
    name     :: Symbol
    Delta    :: Vector{Pair{Symbol,Int}}
    hazard   :: Function
    r        :: Vector{Int}
    type     :: EventType
```

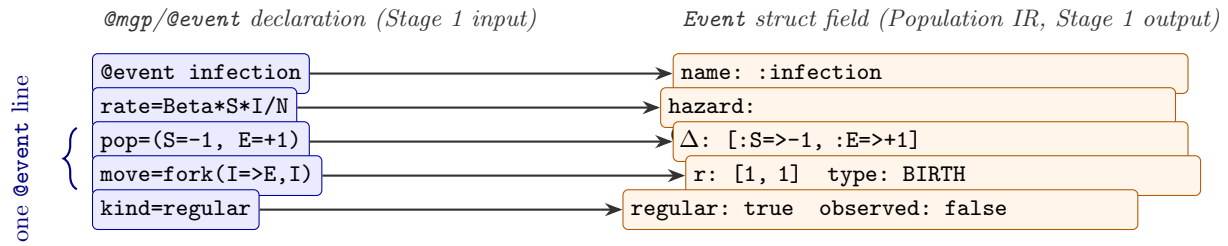


Figure 2: Stage 1's lowering step, made concrete: SEIR's `infection` mark, as written in the DSL (left, blue), and the fields of the `Event` struct it is parsed into (right, orange) — the Population IR the rest of the compiler pipeline consumes. `mgp_macro.jl`'s job is exactly this rewrite: bare model symbols (`S`, `I`, `Beta`) in `rate=` become field accesses on the state/parameter NamedTuples (`x.S`, `x.I`, `theta.Beta`), and the declarative `move=fork(I=>E,I)` notation is decoded into the numeric production vector r^u, event type, and from/into deme indices.

```
    from     :: Int
    into     :: Vector{Int}
    regular  :: Bool
    observed :: Bool
end
```

Field-by-field this is KLI's per-mark data: `hazard` is α_u, `Delta` is Δ_u, `r` is the production vector r^u indexed by deme, `type` is one of the five pure event types of KLI §3.2 (the `@enum` at `mgp.jl:32`), `from` is the ancestral/source deme d_u, `into` the target demes, and `regular` the regular/singular flag. Note what is *absent*: neither the saturation s nor the lineage count ℓ is a field, because both depend on the pruned genealogy at time t and are therefore arguments supplied by the filter at each event, not model data (`mgp.jl:52--57`).

The production vector is *derived from the coloring operator*, not declared. `_production` (`mgp_macro.jl:61--67`) counts emergent slots per deme, and the `:fork` branch of `_decode_move` (`mgp_macro.jl:78--86`) supplies the target list:

```
function _production(targets, demes)
    r = zeros(Int, length(demes))
    for target in targets
        r[_deme_index(target, demes)] += 1
    end
    r
end
[...]
    if op === :fork
        source, first_target = _arrow(ex.args[2])
        targets = _targets(first_target, ex.args[3:end])
        from = _deme_index(source, demes)
        into_demes = copy(targets)
        parent = findfirst(==(source), into_demes)
        isnothing(parent) || deleteat!(into_demes, parent)
        into = unique(_deme_index(d, demes) for d in into_demes)
        return BIRTH, from, collect(into), _production(targets, demes)
```

The `+=` at `mgp_macro.jl:64` is what makes r^u a multiset count rather than a set: `move=fork(I => E, I)` has target list (E, I) and therefore $r^{\text{infection}} = (1, 1)$—one slot for the new exposed child and

one for the still-infectious parent. Lines 82–84 then delete the parent from `into` while leaving it in `targets`, so `into` names the genuinely *new* demes while r^u counts *every* emergent slot, the parent's included. This distinction is exactly what later gives `infection` a nonzero saturation component in its own ancestral deme, hence the InlineSameDeme class of Stage 3. The other operators fall out the same way: `swap(E => I)` gives $r^{\text{progression}} = (0, 1)$ (`mgp_macro.jl:87--93`), `chop(I)` gives $r^{\text{recovery}} = (0, 0)$ with no targets at all (lines 94–96), and `sample(I)` gives $r^{\text{sampling}} = (0, 1)$ (lines 97–100) because a non-destructive sample inserts a leaf slot on a lineage that survives.

Milestone M01 added `audit_model` (`mgp_audit.jl:97--111`) and `validate_model` (`mgp_audit.jl:172`) that walk this IR, confirming every Δ_u, α_u, r^u, and from/into wiring is present and well-typed. These duplicate, deliberately, checks the macro already performs at expansion time—the point being that a hand-written `MGPModel` such as `SEIR_REFERENCE` (`mgp.jl:91--107`), which never passes through `@mgp`, can be validated by the same code path.

Remark 2 (Design decision)**.** A single `Event` struct with an `EventType` enum was retained (not a typed subtype hierarchy), because the closed set of 4 stub functions that dispatch on event semantics is small enough for `if/elseif` branching.

2.3 Stage 2: Production Slots and the Binomial Ratio φ_u (M02)

Given an event with production r^u and a lineage-count state ℓ, the **saturation space** is:

$$S_u(\ell) = \left\{ s \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|} : 0 \leq s_i \leq \min(r_i^u, \ell_i) \; \forall \, i \in \mathcal{D} \right\}. \tag{2}$$

Each saturation s specifies how many of the event's r_i^u emergent lineages in deme i are "filled" by tracked (pruned-tree) lineages.

`enumerate_saturations` realizes (2) as a literal Cartesian product (`mgp_phi.jl:78--80`):

```
D = length(r)
ranges = ntuple(d -> 0:min(r[d], ell[d]), D)
return [collect(Int, s) for s in vec(collect(Iterators.product(ranges...)))]
```

The `ntuple` on line 79 builds one range $0 : \min(r_d^u, \ell_d)$ per deme, which is exactly the per-coordinate constraint of (2), and `Iterators.product` on line 80 forms their Cartesian product, so $|S_u(\ell)| = \prod_i (\min(r_i^u, \ell_i) + 1)$. The two boundary cases KLI's combinatorics require are consequences of the `min`, not special cases: $\ell_d = 0$ collapses deme d's range to $\{0\}$ (a deme with no tracked lineages cannot fill a slot), and $r_d^u = 0$ does likewise (nothing is produced there, so nothing can be saturated there).

For each s, the **binomial ratio** (KLI Eq. 9) is:

$$\varphi_u(s) = Q_u \cdot \prod_{i \in \mathcal{D}} \frac{\binom{n_i - \ell_i}{r_i^u - s_i}}{\binom{n_i}{r_i^u}}, \tag{3}$$

where $n_i = n_i(x)$ is the post-event deme occupancy and $Q_u \in \{0, 1\}$ is the compatibility indicator.

The body of `kli_binomial_ratio` (`mgp_phi.jl:175--185`) is (3) transcribed one factor at a time:

```
any(sd < 0 for sd in s) && return zero(Rational{Int})
phi = Rational{Int}(Q)
for d in 1:D
    denom = safe_binomial(n[d], r[d])
    if denom == 0
```

```
            return zero(Rational{Int})
        end
        numer = safe_binomial(n[d] - ell[d], r[d] - s[d])
        phi *= numer // denom
    end
    return phi
```

The `for d in 1:D` loop at `mgp_phi.jl:177--184` computes exactly the product in (3), one factor per deme: line 182 is the numerator $\binom{n_i-\ell_i}{r_i^u-s_i}$, line 178 the denominator $\binom{n_i}{r_i^u}$. The accumulator is *seeded* with the compatibility indicator at line 176, so Q_u multiplies the product rather than gating it afterwards, which is why $Q_u = 0$ yields an exact zero rather than a rounded one. The `//` on line 183 is Julia's exact rational division, so the return value is a `Rational{Int}` and boundary values 0 and 1 are exactly comparable in tests—no floating-point tolerance appears anywhere in Stages 2–6.

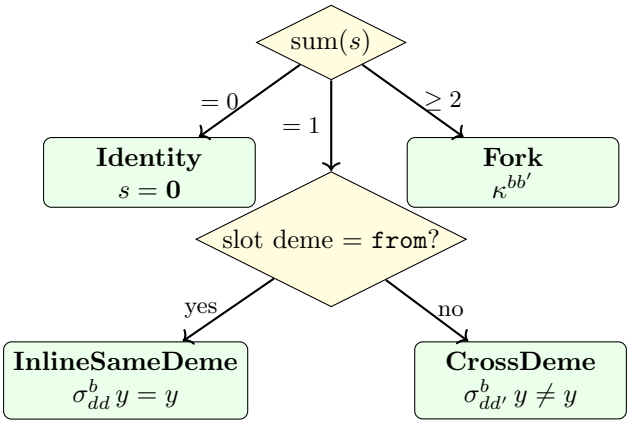
The three degenerate cases are handled by three *different* mechanisms, which is worth separating. $s_i < 0$ is an explicit pre-check (line 175), because $r_i - s_i$ would otherwise exceed r_i and produce a nonzero binomial for a nonsensical negative-count saturation. $s_i > r_i^u$ needs no guard at all: it makes $r_i - s_i < 0$, and $\binom{a}{b} = 0$ for $b < 0$ already. $n_i < r_i^u$ is caught as a zero *denominator* (lines 179–181) and returns 0—the saturation is unreachable given n—rather than raising `DivideError`. All three route through `safe_binomial` (`mgp_phi.jl:103--106`):

```
function safe_binomial(a::Integer, b::Integer)
    (a < 0 || b < 0 || b > a) && return 0
    return binomial(a, b)
end
```

This wrapper is not defensive boilerplate: Julia's `Base.binomial` extends to negative upper arguments by Pascal's rule, so `binomial(-1,1) == -1`—asserted at load time in `mgp_phi.jl:116`—which is the wrong convention for a combinatorial count of slots. `safe_binomial` imposes the KLI convention $\binom{a}{b} = 0$ outside range on all three arguments.

2.4 Stage 3: Full KLI Transition Classification (M03)

Each saturation $s \in S_u(\ell)$ licenses a specific *full coloring transition* $Y \to Y'$ on the pruned genealogy. The function `full_transitions` classifies each s generically from $\mathrm{sum}(s)$ and a comparison of the occupied slot's deme against `event.from` (the ancestral deme):



The decision tree above is not a diagrammatic gloss—it is the literal control flow of `mgp_transitions.jl:230--2`

```
for (i, s) in enumerate(S)
    phi = kli_binomial_ratio(r, s, ell, n; Q = Q)
    total = sum(s)
    transitions[i] = if total == 0
        IdentityTransition(event, s, phi)
    elseif total == 1
        d = findfirst(!=(0), s)
        if d == ancestral
            InlineSameDemeTransition(event, s, phi, d)
        else
            CrossDemeTransition(event, s, phi, ancestral, d)
        end
    else
        slot_demes = Int[]
        for d in eachindex(s)
            for _ in 1:s[d]
                push!(slot_demes, d)
            end
        end
        ForkTransition(event, s, phi, ancestral, slot_demes)
    end
end
```

Line 232's `total = sum(s)` is the diagram's root test, and line 237's `d == ancestral` is its second, where `ancestral = event.from` was bound once at `mgp_transitions.jl:226`. Binding it *once, outside the loop* is the coded form of a mathematical claim: for a given mark, every occupied slot has the same ancestral deme, namely the single source deme d_u, whatever the slot's post-event deme happens to be—so the InlineSameDeme/CrossDeme test is a comparison against a per-event constant, not a per-slot lookup. Note also that φ_u is computed on line 231 for *every* feasible s regardless of class, so classification never alters a weight; it only labels one.

The Fork branch (lines 243–248) expands s into a *multiset* `slot_demes` with one entry per occupied slot, repeated s_d times. That is what keeps $s = (2, 0)$—a within-deme branch point, two slots in deme 1—distinguishable from $s = (1, 1)$, a cross-deme spillover branch point; both have $\mathrm{sum}(s) = 2$ and would be indistinguishable under a set-valued representation.

Finally, `full_transitions` accepts only `BIRTH` and `MIGRATION` events and throws on the rest (`mgp_transitions.jl:212--219`). This is a scope boundary drawn by event *type*, not by whether r^u happens to vanish: KLI compatibility for Death and Sample marks is rate-driven, and those marks are picked up instead by the decay machinery of Stage 6.

Remark 3. Identity and **InlineSameDeme** are distinct at the full-KLI level—they differ in the event-indicator component m—even though both leave the reduced coloring unchanged ($y' = y$ on the deme-only state). This distinction is central to the correctness of m-marginalization (Stage 4).

2.5 Stage 4: Reduction over the Event Indicator m (M04)

The coloring $Y_t(a) = (\mathrm{col}(Y), \mathrm{ctr}(Y))$ has a *color* (deme) component and an *event indicator* component m. KLI's filter equation acts on the reduced, color-only state. The function `reduce_event_indicator` groups the full transitions by their outcome on the reduced `Coloring` and sums:

$$\Phi_u(d, d') = \sum_{\substack{s \in S_u(\ell) \\ \text{reducing to } d \to d'}} \varphi_u(s). \tag{4}$$

(As in the rest of this report, Φ_u is our own name for this m-marginalized sum; KLI performs the same sum but does not give it this symbol.)

The collapsing rule is a four-line dispatch table (`mgp_reduce.jl:115--119`), traced to the actual `swap!` code in `coloring.jl` rather than to the paper's informal notation:

```
reduced_key(t::IdentityTransition) = (:noop, t.event.from)
reduced_key(t::InlineSameDemeTransition) = (:noop, t.deme)
reduced_key(t::CrossDemeTransition) = (:cross, t.ancestral_deme, t.target_deme)
reduced_key(t::ForkTransition) =
    (:fork, t.ancestral_deme, Tuple(sort(t.slot_demes)))
```

Two full transitions collapse together precisely when these keys are equal, so:

- **Identity + InlineSameDeme** → single `(:noop, d)` group. Lines 115 and 116 return the same key because `t.event.from` and `t.deme` are the same deme index for an InlineSameDeme transition (that is what "inline" means, by Stage 3's line 237 test), and because `swap!(y,i,i,b)` is a literal no-op on the `BitSet`: a `delete!` followed by a `push!` of the same lineage id into the same deme's set.
- **CrossDeme** → its own `(:cross, d, d')` group (because `swap!(y,i,j,b)` with $i \neq j$ genuinely changes the coloring), one group per distinct ordered deme pair.
- **Fork** → its own `(:fork, d, ...)` group.

The fork key deserves a note. Equation (4) indexes Φ_u by an ordered pair (d, d'), which is the right index for the `:noop` case ($d \to d$) and the `:cross` case; a branch point, however, is not a pair but an ancestral deme together with a *multiset* of post-event slot demes, which is what line 119's `Tuple(sort(...))` produces. Sorting makes the key order-independent while `Tuple` preserves multiplicity, so two genuinely different Fork saturations of the same event that happened to land in the same demes *would* correctly collapse. No current SEIR/MERS/SI2R event exercises that—each has at most one Fork-classified saturation—but the key is generic rather than a special case of "there is only ever one fork" (`mgp_reduce.jl:45--56`).

The summation itself is `mgp_reduce.jl:162--168`:

```
result = Vector{ReducedTransition}(undef, length(order))
for (i, k) in enumerate(order)
    members = groups[k]
    Phi = sum(t.phi for t in members)
    result[i] = ReducedTransition(k, reduced_kind(members[1]), Phi, members)
end
return result
```

Line 165 is (4): an exact `Rational{Int}` sum of the $\varphi_u(s)$ over precisely those s that share a reduced key. Two implementation choices carry mathematical weight. First, the grouping is driven by an insertion-ordered key vector (`order`, built at lines 152–161) rather than by dictionary iteration order, so the output sequence is deterministic across runs—a prerequisite for the bit-for-bit RNG-order comparisons of Stage 7. Second, the contributing `members` are retained in the result rather than discarded, so every Φ_u can be traced back to the exact set of full transitions that produced it; the audit and explain tooling of the later milestones reads that provenance.

The Chu–Vandermonde identity guarantees the partition-of-unity:

$$\sum_{s \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}} \varphi_u(s) \binom{\ell}{s} = 1, \tag{5}$$

whenever $n_i \geq \{\ell_i, r_i^u\}$ for all i. No compiler function computes (5); it is an identity the compiler's output must satisfy, and it is therefore checked in the test suite rather than implemented. The broadest such check assembles the sum directly from Stage 2's two functions (`test/kli_si2r_test.jl:480--486`):

```
sats = enumerate_saturations(ev.r, ell)
cv_sum = Rational{Int}(0)
for s in sats
    phi = kli_binomial_ratio(ev, s, ell, n)
    cv_sum += phi * prod(binomial(ell[d], s[d]) for d in 1:2)
end
@test cv_sum == 1 // 1
```

Line 484's `prod(binomial(ell[d], s[d]) ...)` is the multinomial weight $\binom{\ell}{s}$ of (5), and line 486 asserts *exact* equality to 1 in $\mathbb{Q}$—not equality within a tolerance. This runs over 200 randomized (ℓ, n) states $\times$ 4 events. A separate M12 property test (`test/kli_properties_test.jl:115--131`, 500 randomized trials) checks the weaker but structurally distinct conservation law $\sum_s \varphi_u(s) = \sum_{\text{groups}} \Phi_u$: that Stage 4 partitions Stage 3's mass without dropping or duplicating any of it.

2.6 Stage 5: Filter IR Classification (M05/M06)

Each `ReducedTransition` is classified as:

Bucket	Condition	KLI role
RegularFlow	`:noop` or `:cross`, `event.regular`	Inflow term: $\alpha_u \cdot \Phi_u$ (non-fork)
SingularFlow	any kind, `!event.regular`	Fixed observed-event-time contribution
OutflowImbalanceTerm	`fork`, `event.regular`	Fork mass resolved by inflow/outflow balance

The table is the function `classify_filter_terms` (`mgp_filter_ir.jl:317--334`), row for row:

```
if event.regular
    for rt in reduced
        if rt.kind == :noop || rt.kind == :cross
            push!(regular, RegularFlow(event, rt, rt.Phi))
        elseif rt.kind == :fork
            push!(outflow_imbalance,
                  OutflowImbalanceTerm(event, rt, rt.Phi, :inflow_outflow_imbalance,
                                       :fork_unobserved_at_regular_time))
        else
            throw(ArgumentError("classify_filter_terms: unrecognized " *
                                "ReducedTransition.kind `$(rt.kind)`"))
        end
    end
else
    for rt in reduced
        push!(singular, SingularFlow(event, rt, rt.Phi))
    end
end
```

The classification reads only two bits of information—`event.regular` (line 317) and `rt.kind` (lines 319, 321)—and nothing about any proposal kernel. That is deliberate: the Filter IR describes the *target* filter structure, so it must be identical whichever of the four proposal strategies of Stage 6 will later be used to sample it. Note too that Φ is copied verbatim from the `ReducedTransition` (`rt.Phi` at lines 320, 323, 332) and never recomputed or rescaled; a term's numeric content is entirely Stage 4's, and Stage 5 contributes only the bucket label plus the `mechanism/reason` tags that record why. Before classifying, the function checks that every full transition's provenance actually names the `event` argument (`mgp_filter_ir.jl:305--311`) and throws otherwise, so one event's reduced transitions cannot be silently filed under another's.

Two limitations are recorded in the source rather than papered over. The `else` branch at lines 330–334 is unreachable for every shipped model: no SEIR, MERS, or SI2R `BIRTH/MIGRATION` event has `regular == false`, and the singular events that do exist are all `SAMPLE`-typed and hence outside `full_transitions`' scope (`mgp_filter_ir.jl:291--297`), so `SingularFlow` is exercised only against a synthetic hand-built `Event` in `test/kli_filter_ir_test.jl`. And the `OutflowImbalanceTerm` constructed at lines 322–324 carries only provenance: its Φ field is the raw reduced sum, not a computed imbalance weight.

Remark 4 (M06 correction)**.** M05's original name "DecayTerm" was renamed to "**OutflowImbalanceTerm**" because this bucket is *not* KLI's $\lambda(t, x, y)$—that is a separate mechanism (sampling hazard + sub-threshold removal). The fork/branch-point mass is resolved by the structural imbalance between the full-hazard regular outflow and the non-fork-only inflow, per the KLI paper's explicit warning: *"Adding either to λ would double-count."* The rename was made structural, not cosmetic: after M06 there is no type or field anywhere in `mgp_filter_ir.jl` whose name contains "decay", and the `mechanism` tag is fixed at `:inflow_outflow_imbalance`, so a later implementer of λ cannot reach for a plausible-looking `FilterSpec.decay` field and sum it in—the field does not exist.

2.7 Stage 6: Decay λ and Driver/Boost (M07)

KLI's **decay term** is:

$$\lambda(t, x, y) = \underbrace{\sum_{u:\,\text{SAMPLE}} \alpha_u(x, \theta)}_{\text{sampling hazard}} + \underbrace{\sum_{u:\,\text{DEATH}} \alpha_u(x, \theta) \cdot \mathbf{1}_{n_{d_u} \le \ell_{d_u}}}_{\text{sub-threshold removal}}, \tag{6}$$

where d_u = event.from is the deme the death event acts on. The two summands of (6) are the two branches of `decay_contribution` (`mgp_decay.jl:247--252` and `262--270`, with the argument-validation guards at lines 253–261 elided):

```
    alpha = event.hazard(x, theta)

    if event.type == SAMPLE
        gate   = one(Rational{Int})
        reason = :sampling_hazard_full
    else # DEATH
        d = event.from
[...]
        elld, nd = ell[d], n[d]
[...]
        gate   = nd <= elld ? one(Rational{Int}) : zero(Rational{Int})
        reason = :sub_threshold_removal
    end
```

```
DecayContribution(event, alpha, gate, alpha * gate, reason)
```

Line 250's unconditional `one(Rational{Int})` is the first summand's missing indicator: a Sample mark contributes its *full* hazard, because the exact likelihood conditions on no sample having occurred at any $t \notin \text{event}(\mathbf{P})$, and no threshold question arises. Line 266's ternary is the indicator $\mathbf{1}_{n_{d_u} \le \ell_{d_u}}$ of the second summand. Two details are load-bearing. First, `gate` is a `Rational`, not a `Bool`, so line 270's `alpha * gate` stays in exact arithmetic whenever the hazard itself is rational—which is how the decay tests hand-verify values. Second, the invariant $\ell_d \le n_d$ is asserted (line 263, throwing if violated) rather than assumed, and under it the condition $n_d \le \ell_d$ collapses to $n_d = \ell_d$ exactly: the decay fires only at the boundary where a removal has no compatible untracked target left (`mgp_decay.jl:44--50`).

The model-level λ is the sum over marks (`mgp_decay.jl:290--295`):

```
total = zero(Rational{Int})
for event in model.events
    event.type in (DEATH, SAMPLE) || continue
    total += decay_contribution(event, x, theta, ell, n).value
end
total
```

The `continue` on line 292 is the reason (6) has exactly two summands and no others: `BIRTH`/`MIGRATION` marks belong to the Filter IR of Stage 5, and `NEUTRAL` marks contribute nothing at all, because a neutral mark never touches the coloring and so has no compatibility condition that could fail (`mgp_decay.jl:73--80`). The loop is generic over `model.events`, so a model with three sampling channels or five removal channels needs no new code.

The **driver** and **boost** decompose the filter's inflow term:

$$\beta_u = \alpha_u \cdot \pi_u, \qquad B_u = \frac{\Phi_u}{\pi_u}, \tag{7}$$

where π_u is the proposal kernel's probability of choosing the reduced outcome. Both are one-liners (`mgp_proposal.jl:96` and `mgp_proposal.jl:123`, with the convenience overload at line 132):

```
driver(alpha::Real, pi::Real) = alpha * pi

boost(Phi::Rational{Int}, pi::Real) = Phi / pi

boost(rt::ReducedTransition, pi::Real) = boost(rt.Phi, pi)
```

These are trivial by design, and naming them is the point: (7) was previously implicit inside eight separately hand-coded kernels, and the composition $\beta_u B_u = \alpha_u \pi_u \cdot (\Phi_u/\pi_u) = \alpha_u \Phi_u$ is what makes the proposal cancel out of the filter's expectation, so every kernel—naive, soft, guided, or hard—must satisfy it however π_u itself is built. The four strategies are represented only as tags (`ProposalStrategy` and its subtypes, `mgp_proposal.jl:60--78`); π_u is not derived generically anywhere in the compiler, by an explicit M00 decision to keep the existing, already-tested hand-designed kernels.

One subtlety in `boost` is easy to get wrong and is documented at `mgp_proposal.jl:108--116`: the π passed in must be the *fully marginalized* probability of the reduced outcome, including any within-outcome degrees of freedom the concrete kernel introduces—for the naive SEIR kernel, the top-level categorical probability of the mark *times* the uniform $1/\ell$ choice of which tracked lineage it acts on—and not merely the top-level entry that a hand-coded kernel happens to store in a variable named `pi`. Using the unmarginalized value would leave $\beta_u B_u \neq \alpha_u \Phi_u$ by exactly the lineage-choice factor.

2.8 Stage 7: Compiled End-to-End Filters (M08/M09)

Milestones M08 and M09 assembled the M01–M07 building blocks into executable, model-specific compiled filters for SEIR and MERS, respectively. Each compiled filter is a structural mirror of the corresponding hand-coded reference filter (`seir_naive.jl`, `mers_naive.jl`)—identical control flow and RNG-call order, but every likelihood term sourced from the generic compiler IR rather than hand-derived closed forms.

The mirroring is literal. Inside `compiled_regular_part!` (`mgp_seir_filter.jl:232`), the two infection branches call back into Stage 3 at run time (`mgp_seir_filter.jl:276--277` and `286`, and `300--302`; the intervening lines 278–285 are an explanatory comment):

```
                    ts = full_transitions(infection, ellpost, npost)
                    Phiid = only(filter(t -> t isa IdentityTransition, ts)).phi
[...]
                    ll += log(Float64(Phiid)) - log(pi_[1])

                    ts = full_transitions(infection, ellpost, npost)
                    Phicr = only(filter(t -> t isa CrossDemeTransition, ts)).phi
                    ll += log(Float64(Phicr)) - log(1 / ellI_pre)
```

Lines 286 and 302 are $\log B_u$ of (7) written out: $\log \Phi_u - \log \pi_u$, with π_u the naive kernel's `pi_[1]` in the no-op branch and its marginalized lineage-choice probability $1/\ell_{\mathsf{I}}$ in the cross-deme branch. The `only(filter(...))` selects a *single* full transition class rather than the `:noop` reduced group, and the reason is a Q_u fact: the naive regular proposal never realizes the InlineSameDeme saturation, so adding its mass—as `reduce_event_indicator` would—would double-count probability the proposal structurally never proposes. This is the one place where regular-versus-singular Q_u gating, deferred through M03–M06, is actually applied.

The decay used by the compiled filter is λ *plus* a proposal-specific correction (`mgp_seir_filter.jl:136--147`):

```
total = Float64(total_decay(model, x, theta, ell, n))
for event in model.events
    event.type == DEATH || continue
    d = event.from
    alphafull = Float64(event.hazard(x, theta))
    nd, elld = n[d], ell[d]
    above = nd > elld
    alphareduced = above ? alphafull * (nd - elld) / nd : 0.0
    leftover = (above ? alphafull : 0.0) - alphareduced
    total += leftover
end
```

Line 136 is exactly (6); lines 137–146 add, per Death mark, the difference between the full hazard and the ℓ-reduced hazard actually offered to the regular proposal. This term is not part of (6) and is not claimed to be: it is bookkeeping specific to a single-particle importance sampler, which has no neighbouring population state to cancel the "a tracked lineage died unobserved" mass against. The source flags its own justification as "a plausible, internally-consistent reading, not a proven one" (`mgp_decay.jl:171--174`), with Gate 5 below as the empirical check.

The 10^{-15} residuals of Figure 3 are the signature of the design: because Stages 2–6 work in exact $\mathbb{Q}$ and only the final `log(Float64(...))` conversions introduce rounding, the compiled and hand-coded filters agree to within a few units in the last place of a `Float64`, not to within a tuned tolerance. Anything larger would indicate an algebraic disagreement rather than an arithmetic one.

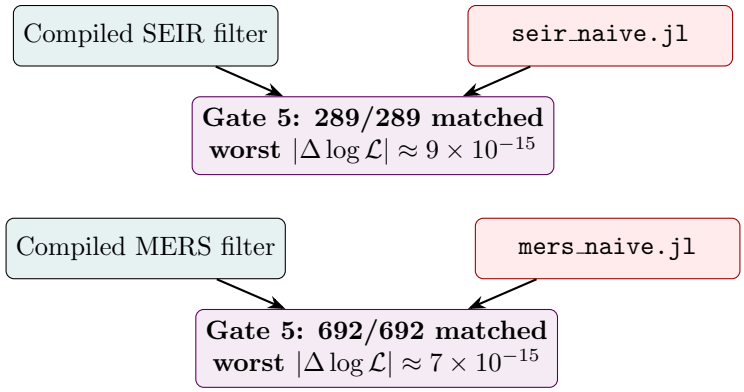


Figure 3: Gate 5 (numerical equivalence) validation. Compiled filters and hand-coded reference filters are run with identical seeds ($N_p = 1$), and their log-likelihoods are compared. All finite comparisons match to floating-point noise.

2.9 Code ↔ Math Correspondence

Table 1 indexes every KLI object named in this section against the equation that defines it here and the Julia code that computes it. Line numbers refer to the files as read at the time of writing; each cited range is the body of the computation, not the surrounding docstring.

Table 1: Correspondence between the KLI objects of this section, the equations that define them, and the Julia code that computes them. Entries with "—" in the equation column are structural rather than numerical: they build or validate the intermediate representation that the numbered equations are then evaluated against.

KLI object	Equation	Julia function	file:line
Population process data $(\alpha_u, \Delta_u, r^u)$	—	`Event`, `MGPModel`	`mgp.jl:59--82`
Deme set $\mathcal{D}$, marks $\mathcal{U}$	—	`@mgp`, `@event`	`mgp_macro.jl:149--186`
Hazard closure $\alpha_u(t, x)$	—	`_rewrite`	`mgp_macro.jl:9--18`
Production vector r^u	—	`_production`, `_decode_move`	`mgp_macro.jl:61--67, 69--106`
IR well-formedness	—	`audit_model`, `validate_model`	`mgp_audit.jl:97--111, 172`
Saturation space $S_u(\ell)$	(2)	`enumerate_saturations`	`mgp_phi.jl:73--81`
Binomial ratio $\varphi_u(s)$	(3)	`kli_binomial_ratio`	`mgp_phi.jl:167--186`
Convention $\binom{a}{b} = 0$ off-range	(3)	`safe_binomial`	`mgp_phi.jl:103--106`
Coloring transitions χ, σ, κ	—	`full_transitions`	`mgp_transitions.jl:210--253`
Class of a saturation	—	classification branch	`mgp_transitions.jl:230--250`
Reduction over m (grouping key)	(4)	`reduced_key`	`mgp_reduce.jl:115--119`
Reduced factor Φ_u	(4)	`reduce_event_indicator`	`mgp_reduce.jl:151--169`
Chu–Vandermonde check	(5)	(identity; no compiler function)	`test/kli_si2r_test.jl:480--486`

continued on next page

Table 1 continued from previous page

KLI object	Equation	Julia function	file:line
Full-to-reduced conservation	(4)	(M12 property P2)	`test/kli_properties_test.jl:115--131`
Filter IR buckets	—	`classify_filter_terms`	`mgp_filter_ir.jl:304--337`
Regular / singular / fork terms	—	`RegularFlow`, `SingularFlow`, `OutflowImbalanceTerm`	`mgp_filter_ir.jl:153--157, 183--187, 241--247`
Per-mark decay	(6)	`decay_contribution`	`mgp_decay.jl:237--271`
Decay $\lambda(t, x, y)$	(6)	`total_decay`	`mgp_decay.jl:287--296`
Driver $\beta_u = \alpha_u \pi_u$	(7)	`driver`	`mgp_proposal.jl:96`
Boost $B_u = \Phi_u / \pi_u$	(7)	`boost`	`mgp_proposal.jl:123, 132`
Proposal families π_u	—	`ProposalStrategy` subtypes	`mgp_proposal.jl:60--78`
Compiled decay (SEIR)	(6)	`compiled_decay`	`mgp_seir_filter.jl:133--148`
Compiled boost at run time	(7)	`compiled_regular_part!`	`mgp_seir_filter.jl:232--302`
Compiled likelihood $\mathcal{L}$	(1)	`compiled_filter_pomp`	`mgp_seir_filter.jl:357`
Compiled MERS filter	(1)	`mers_compiled_regular_part!`	`mgp_mers_filter.jl:304`

Read down the table, the pipeline's shape is visible in the file column alone: one file per stage, each depending only on the one above it, and the two `test/` rows sitting where a mathematical identity—not a computation—is what has to hold.

3 The Mathematics

This section states, in KLI's own terms, the mathematical objects that the compiler manipulates. Every definition below is quoted or transcribed from the reference text, and each is tagged with the section or equation number of that text in which it appears, so that a reader can check the correspondence directly. Where this report uses notation that is *not* KLI's, that is flagged explicitly—see Remark 22 and Table 2.

3.1 From population process to filter equation

3.1.1 The population process $\mathbf{X}_t$

Let $\mathbf{X}_t$ be a continuous-time Markov process on the state space $\mathcal{X}$ with initial density p_0. Following KLI §2.5 ("Jump marks"), the jumps of $\mathbf{X}_t$ are divided into categories "which differ with respect to the changes they induce in a genealogy": there is a countable set $\mathcal{U}$ of *jump marks* such that

$$\alpha(t, x, x') = \sum_{u \in \mathcal{U}} \alpha_u(t, x, x').$$

The *jump mark process* $\mathbf{U}_t$ is the mark of the latest jump as of time t; $\mathbf{U}_t$ is not itself Markov, but $(\mathbf{X}_t, \mathbf{U}_t)$ is.

Two further model-level functions are fixed once and for all.

Definition 5 (Deme occupancy; KLI §2.6)**.** The *deme occupancy function* is $n : \mathcal{D} \times \mathcal{X} \to \mathbb{Z}_{\geq 0}$, where $n_i(x)$ is the number of lineages in deme i when the population is in state x. The elements of $\mathcal{D}$ are the *demes*: subpopulations "within each of which individual lineages are exchangeable."

Definition 6 (Production; KLI §3.3.1)**.** At each event, the lineages descending from a node inserted at that event are said to *emerge* from the event. There is a function $r : \mathcal{U} \times \mathcal{D} \to \mathbb{Z}_{\geq 0}$ such that r_i^u lineages of deme i emerge from each event of mark u; $r^u = (r_i^u)_{i \in \mathcal{D}}$ is the *production* of an event of mark u. KLI are explicit that "the lineages that die as a result of an event do not count in the production but that a parent lineage that survives the event does count."

The Kolmogorov forward equation (KFE) for $\mathbf{X}_t$ is:

$$\frac{\partial v}{\partial t}(t,x) = \int v(t,x')\,\alpha(t,x',x)\,\mathrm{d}x' - \int v(t,x)\,\alpha(t,x,x')\,\mathrm{d}x', \qquad v(0,x) = p_0(x). \tag{8}$$

When deterministic-time events are included (KLI §2.4, "Inclusion of jumps at deterministic times"; sampling at known times $S = \{t_1, t_2, \ldots\}$), the KFE splits into a **regular part** (8) for $t \notin S$ and a **singular part**:

$$v(t,x)\,\mathrm{d}x = \int \tilde{v}(t,x')\,\pi(t,x',\,\mathrm{d}x)\,\mathrm{d}x', \qquad t \in S, \tag{9}$$

where $\tilde{v}$ denotes the left-limit and π is a probability kernel. This regular/singular split is the template for every equation that follows; it recurs verbatim in the general definition of a filter equation (§3.6).

3.1.2 The genealogy, the coloring Y, and inline nodes

A genealogy G is defined by KLI §3.1 as a triple (T, Z, Y), where $T = t(G) \in \mathbb{R}_{\geq 0}$ is the genealogy time, Z specifies the tree structure, and Y gives the coloring.

Definition 7 (Tree structure; KLI §3.1)**.** Let $\mathcal{L}$ be a countable set of labels—referring to samples and/or extant lineages—and let $\mathrm{partit}(\mathcal{L})$ be the collection of all partitions of subsets of $\mathcal{L}$. The tree structure of G is a càdlàg map $Z : [0,T] \to \mathrm{partit}(\mathcal{L})$ that is monotone with respect to partition fineness: $t_1 \leq t_2$ implies $Z_{t_1} \preccurlyeq Z_{t_2}$. An element $b \in Z_t$ is a set of labels; it *represents the branch of the tree that bears the corresponding lineages.*

Definition 8 (Coloring and event indicator; KLI §3.1)**.** If $t \in [0,T]$ and a is the label of any tip or sample node, then

$$Y_t(a) = \big(d(Y_t(a)),\, m(Y_t(a))\big) \;\in\; \mathcal{D} \times \{0,1\},$$

where $d(Y_t(a))$ is the deme in which the lineage of a is located at time t, and $m(Y_t(a)) = 1$ *if there is a node at* t *and* 0 *otherwise.* KLI call $d(Y_t(a))$ the *color* of the branch bearing a at t, and $m(Y_t(a))$ the *event indicator.* Because $a, a' \in b \in Z_t$ implies $Y_t(a) = Y_t(a')$, one may equally regard Y_t as a map $Z_t \to \mathcal{D} \times \{0,1\}$, and KLI "find it useful to think of it in this way most of the time."

Two structural facts from KLI §3.1 are load-bearing for the compiler. First, a genealogy has *inline nodes*: internal nodes with only one descendant. KLI state that these "occur whenever the color changes along a branch, but can also occur without a color-change." Second, $\mathrm{ev}(Z)$ (the discontinuity times of Z) "includes the times of all tip, sample, and branch-point nodes, but excludes root and non-sample inline nodes," so $\mathrm{ev}(Z) \subseteq \mathrm{ev}(G)$. The existence of inline nodes *without* a color change is exactly why a transition with $m = 1$ and no deme change is not the identity—this is the `Identity/InlineSameDeme` distinction of the compiler's transition classifier, and it is a genuine feature of the KLI coloring, not an implementation artifact.

3.1.3 The pruned genealogy $\mathbf{P} = (T, Z, Y)$

Given a genealogy G, KLI §3.4.1 obtain the *pruned genealogy* $P = \mathrm{prune}(G)$ "by first dropping every tip node and then recursively dropping every childless internal node." In a pruned genealogy only internal and sample nodes remain. A pruned genealogy is itself a colored genealogy, characterized by its time $t(P)$ and the functions P^Y and P^Z. Crucially—and this is what makes the filter equation possible—"since it contains within itself all of its past history, the pruned genealogy process $\mathbf{P}_t = \mathrm{prune}(\mathbf{G}_t)$ is Markov."

Definition 9 (Lineage count and saturation; KLI §3.4.2)**.** Let $P = (T, Z, Y)$ be a pruned genealogy and $t \in [0,T]$. Let ℓ_i denote the number of lineages in deme i at time t, and $\ell = (\ell_i)_{i \in \mathcal{D}} \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$. Since ℓ depends only on Y_t, the *lineage-count function* ℓ is defined so that $\ell(Y_t)$ is the vector of deme-specific lineage counts at time t. Similarly, given a node time t, "the number s_i of lineages of deme i *emerging* from all nodes with time t is well defined"; the *saturation function* s is defined so that $s(Y_t) \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$ is the integer vector of deme-specific numbers of emerging lineages at time t.

Note the asymmetry that the compiler must respect: r^u is a property of the *jump mark* (how many lineages emerge in the full population), whereas s is a property of the *pruned genealogy* (how many of those emerging lineages are ancestral to samples). Always $s \leq r^u$ componentwise, with equality only when every emerging lineage happens to be tracked.

3.1.4 The compatibility indicator Q_u

The current report elsewhere writes only "$Q_u \in \{0,1\}$". KLI give it a precise definition, which we reproduce. The motivating observation (KLI §3.4.3) is that "the local structure of P at t is, in general, compatible with only a subset of the possible jumps $\mathcal{U}$. For example, if there is a branch node or a sample node at t, then it is compatible only with birth-type or sample-type jumps, respectively. Similarly, if a lineage moves from deme i to deme i' at t, then u must be either of $i \to i'$ migration type or of a birth type with parent in i and $r^u_{i'} > 0$."

Definition 10 (Compatibility indicator; KLI §3.4.3)**.** Q is the indicator function such that $Q = 1$ if the local genealogy structure—*which is captured by the values of* P^Y *at and just before* t—is compatible with an event of type u, and $Q = 0$ otherwise. Formally,

$$Q_u(y, y') = 1 \iff \begin{array}{l}\text{there is a feasible genealogy } G = (T, Z, Y), \text{ and history } H, \text{ and a } t \in [0, T] \text{ such that,} \\ \text{given } \mathbf{G}_T = G \text{ and } \mathbf{H}_T = H, \text{ we have } U_t = u,\ \tilde{Y}_t = y, \text{ and } Y_t = y'.\end{array}$$

KLI refer to Q as the *compatibility indicator*.

Two things about Definition 10 matter operationally. First, Q_u takes *two* coloring arguments—the left-limit $y = \tilde{Y}_t$ and the value $y' = Y_t$—so it is a predicate on a *transition* of the coloring, not on a coloring. Second, its definition is existential ("there is a feasible genealogy ..."); it is not, as stated, a formula. KLI supply the corresponding *formula* only per-model, by case enumeration, once the chop, swap and fork operators of §3.3 are available. Their SIRS instance (KLI Eq. 29) reads, in full:

$$Q_u(y, y') = \begin{cases} 1, & u = \texttt{Sample},\ \exists\, a, b \text{ s.t. } y' = \chi^{ab} y, \\ 1, & u = \texttt{Trans},\ \exists\, b, b' \text{ s.t. } y' = \kappa^{bb'} y, \\ 1, & u = \texttt{Trans},\ y' = y, \\ 1, & u = \texttt{Trans},\ \exists\, b \text{ s.t. } y' = \sigma^{b} y, \\ 1, & u = \texttt{Recov},\ y' = y, \\ 1, & u = \texttt{Wane},\ y' = y, \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

KLI append the parenthetical: "(Recall that the swap, chop, and fork operators σ, χ, κ are defined in §5.1.)" Equation (10) is therefore the paper's own demonstration that the compatibility predicate is computed by *pattern-matching the coloring transition against the operator forms*—which is precisely what the compiler's transition classifier does.

3.1.5 KLI Theorem 1: conditional likelihood

Theorem 11 (KLI Thm. 1, Eq. 9)**.** *If* $\mathbf{P} = (T, Z, Y)$ *is a given pruned genealogy, define*

$$\varphi_u(x, y', y) \;=\; \frac{\displaystyle\prod_{i \in \mathcal{D}} \binom{n_i(x) - \ell_i(y)}{r^u_i - s_i(y)}}{\displaystyle\prod_{i \in \mathcal{D}} \binom{n_i(x)}{r^u_i}} \;\cdot\; Q_u(y', y), \tag{11}$$

with n *the deme occupancy (KLI §2.6),* r^u *the production (KLI §3.3.1),* ℓ *and* s *the lineage-count and saturation functions (KLI §3.4.2), and* Q *the compatibility indicator (KLI §3.4.3). Then*

$$\mathrm{Prob}\left[\mathbf{P}_T = \mathbf{P} \mid \mathbf{H}_T = H\right] = \mathbf{1}_{\mathrm{ev}(H) \supseteq \mathrm{ev}(\mathbf{P})} \prod_{t \in \mathrm{ev}(H)} \varphi_{U_t}(X_t, \tilde{Y}_t, Y_t).$$

Remark 12 (Argument order)**.** KLI write Eq. 9 as $\phi_u(x, y, y')$ with the binomial ratio evaluated at $\ell(y')$ and $s(y')$ and the compatibility factor as $Q_u(y, y')$; in the product they instantiate it as $\phi_{U_t}(X_t, \tilde{Y}_t, Y_t)$, so KLI's first coloring slot is the *left-limit* and the second is the *value at* t. Equation (11) above carries the primes on the opposite slots but instantiates identically; the two are the same function, and no transposition is intended.

The key insight: the conditional likelihood *factorizes* over event times, with time-local factors φ_u. This is precisely the quantity `kli_binomial_ratio` computes.

3.1.6 KLI Theorem 2: the filter equation

Theorem 13 (KLI Thm. 2, Eq. 10)**.** *If $w(t,x)$ is càdlàg in t, satisfies $w(0,x) = p_0(x)$, and*

$$\begin{aligned} \frac{\partial w}{\partial t}(t,x) &= \sum_u \int w(t,x')\,\alpha_u(t,x',x)\,\varphi_u(x,\tilde{Y}_t,Y_t)\,\mathrm{d}x' - \sum_u \int w(t,x)\,\alpha_u(t,x,x')\,\mathrm{d}x', && t \notin \mathrm{ev}(\mathbf{P}), \\ w(t,x) &= \sum_u \int \tilde{w}(t,x')\,\alpha_u(t,x',x)\,\varphi_u(x,\tilde{Y}_t,Y_t)\,\mathrm{d}x', && t \in \mathrm{ev}(\mathbf{P}), \end{aligned} \tag{12}$$

then the likelihood is $\mathcal{L} = \int w(T,x)\,\mathrm{d}x$.

KLI's own Remark 1 on this theorem is worth recording: "the quantity $w(t,x)$ appearing in Eq. 10 is not itself obviously a probability, except at $t = T$." The compiler therefore may not assume normalization of w at intermediate times.

Comparing (12) with the bare KFE (8):

- The **inflow** term gains the factor φ_u—a genealogy-dependent reweighting.
- The **outflow** term is *unchanged*—it uses the full hazard α_u, not a reduced version.
- This structural asymmetry (full outflow vs. φ_u-weighted inflow) is exactly the mechanism by which the fork/branch-point mass is resolved without appearing in λ.

The last point is visible in KLI's own worked SIRS filter: the regular equation (their Eq. 35) carries inflow terms only for the $s = (0)$ and $s = (1)$ cases of a transmission event, weighted $\varphi\binom{I,\,\ell}{2,\,0} + \ell\,\varphi\binom{I,\,\ell}{2,\,1}$, while the outflow term subtracts the *full* α_u for every mark u including `Trans`. The $s = 2$ (fork) contribution is therefore absent from the regular inflow and is not present in the decay either; it is accounted for solely by that imbalance.

Remark 14 (Where the coloring lives)**.** Theorem 2 conditions on a fully colored pruned genealogy, so $\tilde{Y}_t$ and Y_t are *given*. In practice the coloring is not observed, and KLI's Theorems 4 and 5 introduce an auxiliary coloring process $\mathbf{y}_t$ with proposal kernels q and π_u, yielding a filter equation on $w(t,x,y)$ (their Eqs. 13–14). It is the Theorem-5 form, not the Theorem-2 form, that the compiler ultimately assembles; §3.6 makes the correspondence explicit.

3.2 The binomial ratio: combinatorial meaning

KLI define the binomial ratio (their §3.5, Eq. 7) for $n, r, \ell, s \in \mathbb{Z}_{\geq 0}^{|\mathcal{D}|}$ as

$$\varphi\binom{n,\,\ell}{r,\,s} = \begin{cases} \dfrac{\prod_{i\in\mathcal{D}} \binom{n_i - \ell_i}{r_i - s_i}}{\prod_{i\in\mathcal{D}} \binom{n_i}{r_i}}, & \text{if } \forall i,\ n_i \geq \{\ell_i, r_i\} \geq s_i \geq 0, \\ 0, & \text{otherwise,} \end{cases} \tag{13}$$

and observe that its value lies in $[0,1]$.

The combinatorial interpretation is transparent. At an event of mark u, r_i^u production slots must be filled in deme i. Of the n_i individuals present, ℓ_i are tracked (ancestral to samples) and $n_i - \ell_i$ are untracked. The saturation s_i specifies that s_i of the r_i^u slots are filled by tracked lineages. The numerator counts the ways to fill the remaining $r_i^u - s_i$ "untracked" slots from the $n_i - \ell_i$ untracked individuals; the denominator normalizes over all ways to fill r_i^u slots from n_i individuals. The side condition $n_i \geq \{\ell_i, r_i\} \geq s_i \geq 0$ is not decoration: it is what forces $w \equiv 0$ on states in which the population is too small to support the tracked lineages, e.g. KLI's observation that the SIRS filter "holds only for $I \geq \ell$. For $I < \ell$, it follows from Eq. 32 that $w = 0$."

Remark 15 (Chu–Vandermonde and the branch multiplicity)**.** KLI's Eq. 8 states that, in consequence of the Chu–Vandermonde identity,

$$\sum_{s\in\mathbb{Z}_{\geq 0}^{|\mathcal{D}|}} \varphi\binom{n,\,\ell}{r,\,s}\binom{\ell}{s} = 1$$

whenever $n_i \geq \{\ell_i, r_i\} \geq 0$ for all i. The partition-of-unity is therefore a statement about the *bare* binomial ratio (13), *not* about $\varphi_u = \varphi\binom{n,\,\ell}{r^u,\,s} \cdot Q_u$, and it requires the multiplicity factor $\binom{\ell}{s}$. That factor is combinatorially the number of ways to choose *which* tracked branches carry the s emerging lineages, and it appears explicitly in KLI's own derivations as a sum over branches: their Eq. 34 carries $\sum_{b\in Z_t}$, which collapses to a factor of ℓ in Eq. 35, and their SEIRS derivation carries $\sum_{b\in \mathrm{col}_E(d)}$. Any implementation that sums over s without the branch-choice multiplicity will fail to close; this is what the report's identity (5) and Property P2 test.

3.3 The chop, swap, and fork operators

The compiler's `full_transitions` classifier assigns each candidate transition of the coloring to one of four buckets: `Identity`, `InlineSameDeme`, `CrossDeme`, and `Fork`. These are a computational realization of the operators that KLI introduce in §5.1 ("Chop, swap, and fork operators"), whose stated purpose is that "to specialize the general results to particular models, one needs to be able to speak about events along specific branches of the genealogy." This report has previously named these operators without stating them; we state them now.

Throughout, $z \in \mathrm{partit}(\mathcal{L})$ is the set of branches present at a particular instant, and $y : z \to \mathcal{D} \times \{0,1\}$ is a possible coloring of z, with components $d(y) : z \to \mathcal{D}$ (the color) and $m(y) : z \to \{0,1\}$ (the event indicator). For $i \in \mathcal{D}$, KLI write

$$\mathrm{col}_i(y) \;=\; \{\, b \;:\; d(y)(b) = i \,\}$$

for the set of all branches in y having color i.

Definition 16 (Chop operator χ; KLI §5.1)**.** Suppose $a \in \mathcal{L}$ is a label, $b \in z$ is a branch, $a \in b$, and $i, j \in \mathcal{D}$. Let $b' = b \setminus \{a\}$. If $b' \neq \emptyset$, define

$$\chi^{ab'} z = z \cup \{b'\} \setminus \{b\}, \qquad \text{and let} \qquad \chi^{a\emptyset} z = z \setminus \{\{a\}\}.$$

That is, z and $\chi^{ab'} z$ "are alike except that, in the latter, the label a has been deleted." Moreover, if $b \in \mathrm{col}_i(y)$ and $z' = \chi^{ab'} z$, define $\chi_{ij}^{ab'} y : z' \to \mathcal{D} \times \{0,1\}$ by

$$\chi_{ij}^{ab'} y(b') = (j, 1), \qquad \chi_{ij}^{ab'} y(c) = y(c) \;\text{ for } c \neq b'.$$

If $b = \{a\}$ then $b' = \emptyset$ and $\chi_{ij}^{ab'} y$ is independent of j; in that case KLI write $\chi_i^{a\emptyset} y$ for the common value. In plain language, $\chi_{ij}^{ab'} y$ is the coloring on the branches of z' under which b' is in deme j and all other branches are colored as they are in y.

KLI's operational reading: if (T, Z, Y) is a genealogy with a sample event at t, the sample's label being a, then $Z_t = \chi^{ab'} \tilde{Z}_t$; if $\tilde{Y}_t(b) = i$ and it is a *terminal* sample event, then $Y_t = \chi_i^{a\emptyset} \tilde{Y}_t$; if it is an *inline* sample, the descending lineage being in deme j, then $Y_t = \chi_{ij}^{ab'} \tilde{Y}_t$.

Definition 17 (Swap operator σ; KLI §5.1)**.** Suppose $b \in z$ and $i, j \in \mathcal{D}$. If $b \in \mathrm{col}_i(y)$, define $\sigma_{ij}^b y : z \to \mathcal{D} \times \{0,1\}$ to be another coloring of z that agrees with y everywhere but on branch b. On branch b, "the color is swapped from i to j *and the indicator is set to 1*." That is,

$$\sigma_{ij}^b y(b) = (j, 1), \qquad \sigma_{ij}^b y(c) = y(c) \;\text{ for all } c \neq b.$$

Definition 18 (Fork operator κ; KLI §5.1)**.** Let c, c' be disjoint, nonempty subsets of $\mathcal{L}$ such that $c \cup c' = b \in z$. Let

$$\kappa^{cc'} z = z \cup \{c, c'\} \setminus \{b\},$$

so that $\kappa^{cc'} z$ "is identical to z except in that b has been forked into two branches, c and c'." If $i, j, k \in \mathcal{D}$ and $b \in \mathrm{col}_i(y)$, define $\kappa_{ijk}^{cc'} y$ to be the coloring of $\kappa^{cc'} z$ such that, with $y' = \kappa_{ijk}^{cc'} y$,

$$d(y'(c)) = j, \qquad d(y'(c')) = k, \qquad m(y'(c)) = m(y'(c')) = 1.$$

Thus applying $\kappa_{ijk}^{cc'}$ forks the i-colored branch $c \cup c'$ into a j-colored branch c and a k-colored branch c'. KLI note that "the notation for the fork operator accommodates only binary forks, which will be sufficient for our purposes below," though extension to n-ary forks "would be fairly straightforward."

In each case KLI make the natural componentwise definitions $\chi_{ij}^{ab'} d(y) = d(\chi_{ij}^{ab'} y)$, $\chi_{ij}^{ab'} m(y) = m(\chi_{ij}^{ab'} y)$, and likewise for σ and κ.

Remark 19 (Why `Identity` and `InlineSameDeme` differ)**.** Definition 17 sets the event indicator to 1 *unconditionally*, including when $i = j$. KLI make the point explicitly in the caption of their Fig. 7(D), which "shows an event without an associated change in color. We denote this change with the notation $y' = \sigma_{II}^b y$. In this case, observe that while $d(y') = d(y)$, $m(y') = 1 \neq 0 = m(y)$, whence $y' \neq y$." This is the formal content of the compiler's insistence that `InlineSameDeme` is a distinct bucket from `Identity`: they agree on the color component and disagree on m. The distinction survives only until m is summed out (§3.4), which is exactly why the marginalization must be performed *after* classification, not before.

3.3.1 The reversed operators

KLI §5.1 close by defining reversals $\tilde{\chi}$, $\tilde{\sigma}$, $\tilde{\kappa}$. "It is easy to verify that there are well defined operators" satisfying (their Eq. 27):

Definition 20 (Reversed operators $\tilde{\chi}$, $\tilde{\sigma}$, $\tilde{\kappa}$; KLI §5.1, Eq. 27)**.**

$$
\begin{aligned}
z' = \chi^{a\emptyset} z,\ y' = \chi_i^{a\emptyset} y,\ \{a\} \in \mathrm{col}_i(y) &\iff z = \tilde{\chi}^{a\emptyset} z',\ y = \tilde{\chi}_i^{a\emptyset} y',\ a \notin \bigcup z', \\
z' = \chi^{ab} z,\ y' = \chi_{ij}^{ab} y,\ b \cup \{a\} \in \mathrm{col}_i(y) &\iff z = \tilde{\chi}^{ab} z',\ y = \tilde{\chi}_{ij}^{ab} y',\ b \in \mathrm{col}_j(y'),\ a \notin \bigcup z', \\
y' = \sigma_{ij}^{b} y,\ b \in \mathrm{col}_i(y) &\iff y = \tilde{\sigma}_{ij}^{b} y',\ b \in \mathrm{col}_j(y'), \\
y' = \kappa_{ijk}^{cc'} y,\ c \cup c' \in \mathrm{col}_i(y) &\iff y = \tilde{\kappa}_{ijk}^{cc'} y',\ c \in \mathrm{col}_j(y'),\ c' \in \mathrm{col}_k(y').
\end{aligned}
\tag{14}
$$

The reversed operators are not a curiosity. In the inflow term of a filter equation one integrates against $\tilde{w}(t, x', y')$ where y' is the coloring that *maps to* the current coloring, i.e. the pre-image. KLI's worked SEIRS singular equation (their Eq. 39) accordingly reads

$$
w(t,x,d,m) = \int \tilde{w}\big(t, x', \tilde{\chi}_I^{a\emptyset} d,\ \tilde{\chi}^{a\emptyset} m\big)\, \alpha_{\texttt{Sample}}(t, x', x)\, \varphi\begin{pmatrix} (E,I),\ (\ell_E, \ell_I) \\ (0,1),\ (0,0) \end{pmatrix} \mathrm{d}x', \qquad t \in S_0,
$$

and analogously with $\tilde{\chi}_{II}^{ab}$ at inline samples and $\tilde{\kappa}_{IIE}^{bb'}$ at branch points. Consequently the compiler's `from`/`to` deme bookkeeping runs in the direction *opposite* to the operator's own subscript order: an inflow entry for target coloring y is indexed by the source y' recovered by the reversed operator.

Remark 21 (Operators versus buckets: not a bijection)**.** The four classifier buckets are *not* in bijection with the three KLI operators. `InlineSameDeme` and `CrossDeme` are the $i = j$ and $i \neq j$ cases of σ_{ij}^{b}; `Fork` is $\kappa_{ijk}^{cc'}$; `Identity` corresponds to the $y' = y$ cases that appear on their own lines in KLI's Q_u table (10)—for `Trans`, `Recov`, and `Wane`. There is no bucket for the chop operator χ, because chops arise only at sample events, which are observed-event times and are handled by the singular part of the filter equation (`SingularFlow` in the Filter IR) rather than by the regular transition classification.

3.4 Summing over the event indicator m

The coloring carries two components, $y = (d, m)$. KLI's general filter equations (their Eqs. 13–14) are stated in terms of y, and their worked examples proceed by making the two components explicit and then *marginalizing over* m. Because this report leans on that step throughout, it is worth exhibiting KLI's own execution of it.

For the one-deme SIRS model, KLI write "recalling that a coloring has two components, i.e., $y = (d, m)$, where $d = d(y)$ is the deme, or color, and $m = m(y)$ is the event indicator (cf. §3.1), we can make these explicit in writing $w = w(t, x, d, m)$. In the case of the SIRS model, however, there being only one deme, the dependence of w on d is trivial." The singular part then becomes their Eq. 30:

$$
w(t,x,m) = \sum_u \sum_{m'} \int \tilde{w}(t, x', m')\, \alpha_u(t, x', x)\, \varphi_u(t, x', x, m', m)\, \mathrm{d}x', \qquad t \in \mathrm{ev}(Z). \tag{15}
$$

Equation (15) is the *pre-marginalization* form: w still carries m. KLI then specialize it using the cases of their Q_u table and state the marginalization in a single unnumbered sentence:

$$
\text{"Defining } w(t,x) = \sum_m w(t,x,m),\ \text{we have } \ldots\text{"} \tag{16}
$$

whereupon their Eq. 31 is the m-free singular filter equation. The same move is made three more times in the paper: for the SIRS regular part ("Again, we sum over m, obtaining" their Eq. 35), and twice in the SEIRS example, where the multi-deme version is written out as

$$
w(t,x,d) \ = \ \sum_m w(t,x,d,m)
$$

with the accompanying text "as in §5.2, we sum over m" and "as in §5.2, we then sum over m, obtaining."

Two consequences. First, in the SIRS regular case the m-sum combines with the branch sum $\sum_{b \in Z_t}$ of KLI's Eq. 34, and their Eq. 35 emerges with the transmission inflow coefficient

$$
\varphi\begin{pmatrix} I,\ \ell \\ 2,\ 0 \end{pmatrix} \ + \ \ell\, \varphi\begin{pmatrix} I,\ \ell \\ 2,\ 1 \end{pmatrix},
$$

in which the ℓ is precisely the branch-choice multiplicity $\binom{\ell}{s}$ of Remark 15. The $s = 2$ term—the fork—does not appear. Second, the surviving terms are exactly the grouping that the compiler's `reduce_event_indicator` performs: the $y' = y$ and σ^b_{ii} transitions merge (they differ only in m), while σ^b_{ij} with $i \neq j$ and $\kappa^{cc'}_{ijk}$ do not merge with anything, because they change the surviving d-component.

Remark 22 (Provenance of the symbol Φ_u)**.** The summation of §3.4 is genuine KLI content: the paper performs it four times, in §5.2 and §5.3, and its results are KLI's own Eqs. 31, 35, 40 and 41. What KLI never do is assign the *result* of that summation a name or a symbol. The marginalization is introduced in running prose ("Defining $w(t,x) = \sum_m w(t,x,m)$ …"), the defining display is not numbered, and no letter is attached to the m-summed φ-coefficient anywhere in the paper.

The symbol Φ_u used in this report—from Section 2 onward, in (4), in (7), and in the Filter IR—is therefore **this project's own notation**, not verbatim KLI notation. It names the m-marginalized, reduced-coloring transition weight, i.e. the coefficient that stands where φ_u stood before the m-sum was taken.

Separately, and independently: the KLI paper *does* use capital Φ, in two places, for things entirely unrelated to the above. In its §1, Eq. 1, Φ denotes *a genealogical tree* in the conventional two-stage phylogenetic decomposition $\mathcal{L}(D,E) = f(S \mid \Phi, E)\, f(\Phi \mid D)$ integrated $\mathrm{d}\Phi$. In its §2.1 ("Notation"), Φ_t is a generic placeholder for an arbitrary stochastic process, used only to define the left- and right-limit conventions $\tilde{\Phi}_t = \lim_{s\uparrow t} \Phi_s$, $\Phi_t = \lim_{s\downarrow t} \Phi_s$, the càdlàg/càglàd terminology, and the embedded-chain notation $\hat{\Phi}_k$. A reader should therefore *not* read Φ_u in this report as corresponding to any Φ in the original paper. We have found no other occurrence of capital Φ in KLI, and in particular none in Appendix B.

To be precise about what is and is not KLI's: KLI *do* name the un-summed weight, φ_u (their Eq. 9); they *do* name the generic boost, B (their Eqs. B2–B3); and they *do* name the Theorem-5 boost, $\Psi_u = \varphi_u/\pi_u$. The single object in this chain that KLI leave unnamed is the m-summed φ_u. That, and only that, is what Φ_u supplies. The operation is KLI's; the letter is ours.

3.5 The decay term $\lambda(t,x,y)$

The decay term in the filter equation arises from two sources:

1. **Sampling hazard**: every SAMPLE-type event contributes its full hazard α_u unconditionally to λ, regardless of whether sampling is destructive ($r = 0$) or non-destructive ($r > 0$).
2. **Sub-threshold removal**: a DEATH-type event in deme d contributes $\alpha_u \cdot \mathbf{1}_{n_d \le \ell_d}$ to λ. When $n_d > \ell_d$, the death can occur without killing a tracked lineage, so it contributes to the regular outflow instead. When $n_d = \ell_d$, every individual is tracked, so any death necessarily removes a tracked lineage—a catastrophic event that the filter absorbs as exponential decay.

KLI exhibit exactly this pattern in their SEIRS example. Their Eq. 47 reads, in full,

$$\lambda(t,x,d) \;=\; \int \alpha_{\texttt{Sample}}(t,x,x')\,\mathrm{d}x' \;+\; \int \alpha_{\texttt{Recov}}(t,x,x')\,\mathbf{1}_{I \le \ell_I}\,\mathrm{d}x', \tag{17}$$

and the *complementary* indicator appears in their driver: the $\alpha_{\texttt{Recov}}$ term of their Eq. 45 carries $\mathbf{1}_{I>\ell_I}$. The recovery hazard is thus split exhaustively and without overlap between β and λ according to whether the deme has untracked individuals to spare. KLI also note, of the one-deme SIRS filter, "the presence in Eq. 36 of the decay term proportional to ψ"—the sampling rate—confirming source (i).

Remark 23 (Scope of (6))**.** KLI's Eq. 47 is a worked instance for one model, and Appendix B's Definition leaves λ an arbitrary measurable function $\lambda : \mathbb{R}_{\ge 0} \times \mathcal{X} \to \mathbb{R}$. The generic sum (6) over *all* SAMPLE- and DEATH-type marks of a model, which `total_decay` implements, is therefore a generalization of the pattern KLI exhibit rather than the restatement of a KLI theorem. Note also that KLI write the argument list as $\lambda(t,x,d)$—on the *reduced* coloring—consistent with λ being read off after the m-sum of §3.4.

Remark 24 (Decay vs. outflow imbalance)**.** The fork/branch-point mass (classified as `OutflowImbalanceTerm` in the Filter IR) is *not* part of λ. It is resolved by the structural imbalance between the full-hazard outflow and the `RegularFlow`-only inflow in the assembled filter equation—not by a separate rate-driven mechanism. The source-level evidence is KLI's Eq. 35: its regular transmission inflow retains only the $s = 0$ and $s = 1$ coefficients, its outflow subtracts the full $\sum_u \alpha_u$, and its λ (Eq. 47, and the ψI term of Eq. 36) contains no branch-point contribution.

3.6 Driver, boost, and decay

The report's (7) is not an ad hoc factorization; it instantiates the general object that KLI define in Appendix B.

Definition 25 (Filter equation; KLI Appendix B1, Eqs. B2–B3)**.** Let $\mathbf{X}_t$ be a continuous-time Markov process with KFE

$$\frac{\partial u}{\partial t}(t,x) = \int u(t,x')\,\beta(t,x',x)\,\mathrm{d}x' - \int u(t,x)\,\beta(t,x,x')\,\mathrm{d}x'.$$

Suppose $B : \mathbb{R}_{\geq 0} \times \mathcal{X}^2 \to \mathbb{R}_{\geq 0}$ and $\lambda : \mathbb{R}_{\geq 0} \times \mathcal{X} \to \mathbb{R}$ are measurable, and let $S \subset \mathbb{R}_{\geq 0}$ be locally finite. Then the system

$$\frac{\partial w}{\partial t}(t,x) = \int w(t,x')\,\beta(t,x',x)\,B(t,x',x)\,\mathrm{d}x' - \int w(t,x)\,\beta(t,x,x')\,\mathrm{d}x' - \lambda(t,x)\,w(t,x), \qquad t \notin S, \tag{18}$$

$$w(t,x) = \int \tilde{w}(t,x')\,\beta(t,x',x)\,B(t,x',x)\,\mathrm{d}x', \qquad t \in S, \tag{19}$$

"is called the filter equation driven by β, with boost B, decay λ, and observed event times S." KLI add: "without important confusion, the process $\mathbf{X}_t$ can also be said to drive the filter equation. Eq. B2 is the regular part of the filter equation; Eq. B3 is known as the singular part."

KLI's Remark 4 records the degenerate case: "trivially, a Kolmogorov forward equation is itself a filter equation with boost 1, decay 0, and $S = \emptyset$." Comparing (18)–(19) with the bare KFE (8) and its singular companion (9) makes the whole architecture legible: the compiler's job is to compute, from a model specification, the triple (β, B, λ) together with the event-time set S.

The bridge from Definition 25 back to the genealogical theorems is KLI's Theorem 5, which supplies the driver and boost explicitly. Given proposal kernels q and π_u as in their Theorem 4, Theorem 5 sets

$$\beta_u(t,x,x',y,y') = \alpha_u(t,x,x')\,\pi_u(t,x,x',y,y'), \qquad \Psi_u(t,x,x',y,y') = \frac{\varphi_u(x',y,y')}{\pi_u(t,x,x',y,y')}, \tag{20}$$

and shows that the resulting $w(t,x,y)$ gives the likelihood of the obscured genealogy as $\mathcal{L} = \sum_{y \in \mathcal{Y}_T(Z)} \int w(T,x,y)\,\mathrm{d}x$. KLI comment on the kernels: "the importance-sampling kernels π represent the probabilities of changing color on a particular branch (or not doing so), contingent on the population event… One has wide latitude in the choice of these kernels and by tuning them, one can attempt to minimize Monte Carlo variance."

Placing (7) beside (20) gives the exact correspondence:

This report	**KLI**	**Where in KLI**
$\beta_u = \alpha_u \cdot \pi_u$	$\beta_u = \alpha_u \pi_u$	Thm. 5, verbatim
$B_u = \Phi_u / \pi_u$	$\Psi_u = \varphi_u / \pi_u$	Thm. 5; and B in Eqs. B2–B3
λ (6)	$\lambda(t,x,d)$	Appendix B1 (generic); Eq. 47 (SEIRS instance)

The only discrepancy is the numerator of the boost: KLI's Theorem 5 carries the un-summed φ_u because their w still carries the full coloring y, whereas the compiler assembles the filter on the reduced (m-summed) coloring and so carries Φ_u in that slot— the object named in Remark 22. KLI reach the same reduced form themselves in §5.2–5.3; they simply do not name the numerator.

3.7 The SMC algorithm

The filter equation (12), in the driver/boost/decay form of Definition 25, is solved numerically by a sequential Monte Carlo (SMC) algorithm. KLI give it in Appendix B4 ("Solving filter equations by sequential Monte Carlo") as **Algorithm B1**, justified by their Lemma 3, which exhibits $w(t,x) = \int_0^\infty v\,u(t,x,v)\,\mathrm{d}v$ for a u satisfying a KFE in an augmented state. KLI describe the augmented process directly: "the driver $\mathbf{X}_t$ has KFE Eq. B1… [at each jump the weight is multiplied] by the multiplicative factor $B(t,x,x') \geq 0$. Between jumps, $\mathbf{V}_t$ decays deterministically and exponentially." Following KLI, set

$$A(x) = \int \beta(x,x')\,\mathrm{d}x', \qquad \pi(x,\,\mathrm{d}x') = \frac{\beta(x,x')}{A(x)}\,\mathrm{d}x'.$$

Note the asymmetry between lines 6 and 10, which is KLI's, not ours: at an *observed* event time the weight is multiplied by $A(x_j)\,B$—the total hazard times the boost, because an event is *known* to occur—whereas at a spontaneously drawn jump the weight is multiplied by B alone and the exponential decay factor $e^{-\lambda\Delta}$ is applied over the sojourn. This is the computational shadow of the regular/singular split of (18)–(19).

The boost $B_u = \Phi_u / \pi_u$ is the importance-sampling correction factor: it re-weights the proposal kernel π_u to match the target Φ_u.

Algorithm 1: Sequential Monte Carlo Filter (KLI Appendix B4, Algorithm B1)

Input: Initial time t_0, final time T, observed event times $S = \{t_1, \ldots, t_K\}$ with $t_0 \le t_1 < \cdots < t_K < t_{K+1} \coloneqq T$, event rates β, boost B, decay λ, initial distribution p_0, ensemble size J

Output: Unbiased Monte Carlo likelihood estimate $\hat{\mathcal{L}}$

1. Initialize: $x_j \sim p_0$, $w_j \leftarrow 1$ for $j = 1, \ldots, J$
2. **for** $k = 0, \ldots, K$ **do**
3. **for** $j = 1, \ldots, J$ **do**
4. $t \leftarrow t_k$
5. **if** $k > 0$ **then** *// Singular part (Eq. B3)*
6. $x'_j \sim \pi(x_j, \cdot)$; $w_j \leftarrow w_j \cdot A(x_j) \cdot B(t, x_j, x'_j)$; $x_j \leftarrow x'_j$
7. **while** $t < t_{k+1}$ **do** *// Regular part (Eq. B2)*
8. $\Delta \sim \mathrm{Exp}(A(x_j))$
9. **if** $t + \Delta < t_{k+1}$ **then**
10. $x'_j \sim \pi(x_j, \cdot)$; $w_j \leftarrow w_j \cdot B(t{+}\Delta, x_j, x'_j) \cdot e^{-\lambda(x_j)\,\Delta}$; $x_j \leftarrow x'_j$; $t \leftarrow t + \Delta$
11. **else**
12. $w_j \leftarrow w_j \cdot e^{-\lambda(x_j)\,(t_{k+1} - t)}$; $t \leftarrow t_{k+1}$
13. $\hat{\mathcal{L}} \leftarrow \frac{1}{J} \sum_j w_j$

3.8 Notation strictly per KLI

Table 2 lists every symbol used in Sections 2–3 of this report, with the KLI section or equation in which it is defined. Rows marked "*not KLI*" are this project's own notation; the reader should not expect to find them in the reference text.

Table 2: Provenance of every symbol used in Sections 2–3. Section and equation numbers refer to King, Lin, & Ionides. Each entry was checked against the reference text; entries for which no KLI source exists are marked *not KLI* rather than approximated.

Symbol	Source in KLI	Meaning
Processes and spaces		
$\mathbf{X}_t$	§2.2, §2.3	Population process on $\mathcal{X}$
$\mathcal{X}$	§2.2	State space of the population process
$\mathbf{G}_t$	§3.3	Genealogy process
$\mathbf{P}_t$	§3.4.1	Pruned genealogy process (Markov)
$\mathbf{H}_t$	§2.8	History process
$\mathbf{U}_t$	§2.5	Jump mark process
$\mathbf{y}_t$	Thm. 4	Auxiliary (proposed) coloring process
$\mathcal{U}$	§2.5	Countable set of jump marks
$\mathcal{D}$	§2.6	Index set of demes
$\mathcal{L}$	§3.1	Countable set of lineage/sample labels
$\mathrm{ev}(\cdot)$	§2.8, §3.1	Set of (jump / genealogical event) times
Model-level functions		
α, α_u	§2.3, §2.5	Total and mark-specific transition rates
$n_i(x)$	§2.6	Deme occupancy
r^u, r_i^u	§3.3.1	Production of an event of mark u

continued on next page

Table 2 continued from previous page

Symbol	Source in KLI	Meaning
p_0	§2.3	Initial density of $\mathbf{X}_0$
θ	*not KLI*	Parameter vector; KLI leave rates parameterized implicitly
Genealogical quantities		
T, Z, Y	§3.1	Genealogy time, tree structure, coloring
d, m	§3.1	Color (deme) and event indicator components of Y
$\mathrm{col}_i(y)$	§5.1	Branches of y having color i
ℓ, ℓ_i	§3.4.2	Lineage-count function
s, s_i	§3.4.2	Saturation function
$Q_u(y, y')$	§3.4.3; Eq. 29	Compatibility indicator (Definition 10)
$\varphi\binom{n,\ell}{r,s}$	Eq. 7	Binomial ratio. *Caveat:* KLI's Eq. 7 is a bare parenthesized 2×2 array with no leading letter; the φ rendered by this report's `\binratio` macro is decorative
Eq. (5)	Eq. 8	Chu–Vandermonde partition of unity. Holds for the bare ratio, *not* for φ_u with Q_u attached
$\varphi_u(x, y', y)$	Eq. 9 (Thm. 1)	Binomial ratio $\times\, Q_u$
$\chi_{ij}^{ab'}$, $\chi_i^{a\emptyset}$	§5.1	Chop operator (Definition 16)
σ_{ij}^{b}	§5.1	Swap operator (Definition 17)
$\kappa_{ijk}^{cc'}$	§5.1	Fork operator (Definition 18)
$\tilde{\chi}$, $\tilde{\sigma}$, $\tilde{\kappa}$	Eq. 27	Reversed operators, used in inflow terms
Filter-equation quantities		
$v(t, x)$	§2.3 (Eq. 4)	KFE solution (a probability density)
w	Thm. 2 (Eq. 10); Thm. 5 (Eqs. 13–14); Eqs. B2–B3	Filter solution; not a probability except at $t = T$ (KLI Remark 1)
$\mathcal{L}$	Thm. 2, Thm. 5	Likelihood
β, β_u	Eq. B1; Thm. 5	Driver
B	Eqs. B2–B3	Boost (generic)
Ψ_u	Thm. 5	Boost, instantiated as φ_u/π_u
π_u	Thm. 4, Thm. 5	Coloring proposal kernel
q	Thm. 4	Initial-coloring proposal kernel
λ	Appendix B1; Eq. 47	Decay
S	§2.4; Appendix B1	Observed / deterministic event times
$A(x)$, $\pi(x, \mathrm{d}x')$	Algorithm B1	Total proposal hazard and normalized jump kernel
J, $\hat{\mathcal{L}}$	Algorithm B1	Ensemble size; MC likelihood estimate
Φ_u	*not KLI*	The m-summed transition weight. The *summation* is KLI's (§5.2, §5.3; their Eqs. 31, 35, 40, 41) but they never name its result. KLI's own uses of capital Φ (§1 Eq. 1; §2.1) are unrelated—see Remark 22
Compiler-internal names		
`Identity`, `InlineSameDeme`, `CrossDeme`, `Fork`	*not KLI*	Classifier buckets; realize the $y' = y$, σ_{ii}^b, σ_{ij}^b and $\kappa^{cc'}$ cases of KLI §5.1 / Eq. 29
`RegularFlow`, `SingularFlow`	*not KLI*	Filter-IR buckets; correspond to KLI's regular (Eq. B2) and singular (Eq. B3) parts

continued on next page

Table 2 continued from previous page

Symbol	Source in KLI	Meaning
`OutflowImbalanceTerm`	*not KLI*	Filter-IR bucket for fork mass. No KLI name exists; KLI resolve it implicitly by the inflow/outflow asymmetry of their Eq. 35

4 Why This Is Not Circular Logic

A natural concern with any self-contained implementation of a novel mathematical framework is circularity: if the theory, its implementation, and its reference tests all come from the same project, what guards against a self-consistent but wrong system?

The verification of PhyloPOMP.jl addresses this concern through five layers of evidence, each independent of the others: three internal cross-checks, one literature-external check that breaks the circularity for a specific, well-defined slice of the compiler (Layer 4), and one development-external generalization-plus-falsifiability check on a model never used during the compiler's development (Layer 5).

4.1 Layer 1: The KLI theory is proven, not assumed

KLI Theorems 1–4 are proven from probability axioms in the reference paper. The proofs use:

- The exchangeability of lineages within each deme (a standard assumption, shared by coalescent and birth-death approaches).
- The Markov property of the joint process $(\mathbf{X}_t, \mathbf{U}_t, \mathbf{G}_t)$.
- The conditional independence of genealogical choices at distinct event times, given the history.

The binomial ratio (11) is derived, not postulated—it is the unique expression consistent with uniform-within-deme selection of parents, offspring, and migrants. The filter equation (12) follows by substituting this into the KFE.

4.2 Layer 2: Generic compiler verified against hand-derivation

The M02–M04 compiler stages (`enumerate_saturations`, `kli_binomial_ratio`, `full_transitions`, `reduce_event_indicator`) were verified *term-by-term* against the hand-derived MERS reference document (`mers_filter_suite.tex`), using exact $\mathbb{Q}$-arithmetic.

Event mark	r^u	# saturations checked	Match
TCC (`transmission_cc`)	$(2, 0)$	3/3	exact
THH (`transmission_hh`)	$(0, 2)$	3/3	exact
THC (`transmission_hc`)	$(1, 1)$	4/4	exact
TCH (`transmission_ch`)	$(1, 1)$	4/4	exact
SEIR `infection`	$(1, 1)$	4/4	exact
SEIR `progression`	$(0, 1)$	2/2	exact
RC, RH, SC, SH	$(0, 0)$	1/1 each	exact

While `mers_filter_suite.tex` is itself project-authored (not an external source), its formulas were hand-derived from the KLI paper independently of the compiler code—a second, structurally different path to the same numbers.

4.3 Layer 3: End-to-end numerical equivalence (Gate 5)

The compiled SEIR and MERS filters (M08/M09) were compared against independently hand-coded reference filters (`seir_naive.jl`, `mers_naive.jl`) using seed-matched $N_p = 1$ particle-filter runs:

Model	Finite pairs	Matches	Worst $\lvert\Delta \log \mathcal{L}\rvert$
SEIR (permanent test)	289	289	$\approx 8.9 \times 10^{-15}$
SEIR (development sweep)	1913	1913	floating-point noise
MERS (permanent test)	692	692	$\approx 7.1 \times 10^{-15}$
MERS (development sweep)	2903	2903	$\approx 1.4 \times 10^{-14}$

The compiled and reference filters are structurally independent implementations of the same theory—neither consumes the other's code. However, both are project-authored, so this layer alone does not break circularity.

4.4 Layer 4: The Kingman–Moran external check (M12b)

This is the layer that escapes circularity. The test checks the compiler's φ_u formula against a *textbook result that predates the KLI project entirely*: the Kingman coalescent rate for the Moran model.

4.4.1 The standard result (derived from scratch, not from the project)

In a Moran model with population size N, the probability that a specific pair of tracked lineages $\{i, j\}$ is the causally-involved pair at a given birth-death step is:

$$P(\text{pair } \{i,j\} \text{ is involved}) = \frac{1}{\binom{N}{2}}. \tag{21}$$

Summing over the $\binom{\ell}{2}$ tracked pairs:

$$P(\text{any tracked pair coalesces}) = \frac{\binom{\ell}{2}}{\binom{N}{2}}.$$

This is an **exact finite-N identity**, not a large-N diffusion approximation—it is the combinatorial fact that Kingman's theorem is a limit of.

4.4.2 Mapping to the compiler's output

MERS's TCC mark (`transmission_cc`, $r = (2, 0)$, both production slots in deme C) at the full saturation $s = (2, 0)$ yields a `ForkTransition`. Evaluating the compiler's `kli_binomial_ratio`:

$$\varphi_{\text{fork}} = \frac{\binom{I_C - \ell_C}{0}}{\binom{I_C}{2}} \cdot \frac{\binom{I_H - \ell_H}{0}}{\binom{I_H}{0}} = \frac{1}{\binom{I_C}{2}},$$

independent of ℓ_C, ℓ_H, and I_H entirely. This is *exactly* the "this specific pair" probability (21) with $N = I_C$.

4.4.3 Numerical verification

`test/kli_kingman_moran_test.jl` performs 2507 exact $\mathbb{Q}$-arithmetic assertions:

Part	What it checks	Assertions
0	Brute-force $\binom{N}{2}$ ground truth, $N \leq 14$	390
1	$\varphi_{\text{fork}} = 1/\binom{N}{2}$, independence from other deme	808
2	`full_transitions`' Fork reproduces the same φ	400
3	$\alpha \cdot \binom{\ell}{2} \cdot \varphi_{\text{fork}} = \alpha \cdot \binom{\ell}{2} / \binom{N}{2}$	902
4	Boundary states ($N = 0, 1$; $\ell = N = 2$)	7

Every comparison is **exact** $\mathbb{Q}$ equality (`==`, not $\approx$). Zero discrepancies were found.

4.4.4 Why this genuinely helps

The target formula $\binom{\ell}{2}/\binom{N}{2}$ is derived from elementary, textbook, pre-project probability (symmetric population, exchangeability, disjoint-event additivity) and cross-checked by brute-force enumeration that calls *none of this project's code*. Agreement here rules out the specific failure mode M11 flagged—the entire project inventing a self-consistent but wrong theory—for the one piece checked, because the Moran formula did not come from inside the project at all.

4.4.5 Honest scope

This external check covers exactly one structural pattern—the "two production slots, one deme" Fork case, at TCC and THH—not the driver, decay, reduction, cross-deme, or SEIR machinery. The full verification picture is:

4.5 Layer 5: SI2R — a model external to development

Layer 4 breaks circularity by reaching outside the literature for a single formula. Layer 5 attacks a different failure mode. The concern it addresses is not "is the φ_u formula right?" but "is the compiler *general*, or has it been quietly fitted to the two models it was developed against?"—and, sharper still, "would these tests notice if it had not been?" The evidence is accordingly of a different kind: a generalisation check against a reference authored outside this project's development, followed by a direct falsification experiment that measures whether the check is capable of failing.

Throughout this subsection, SI2R's two demes are written L and H with occupancies I_L, I_H and lineage counts ℓ_L, ℓ_H; its two migration *marks*, confusingly but unavoidably named `L` and `H` in the source model, are always set in `typewriter`.

4.5.1 The reference (authored outside this project's development)

SI2R is a two-deme superspreading epidemic model: compartments (S, I_L, I_H, R), demes $\mathcal{D} = \{\mathsf{L}, \mathsf{H}\}$, with I_L the low-rate infectious class and I_H the high-rate (superspreader) class. It carries nine jump marks:

Mark	**Type**	r^u	**Hazard**
TL	BIRTH	$(2,0)$	$\beta S I_L/N$
TH	BIRTH	$(1,1)$	$\kappa\beta S I_H/N$
`L`	MIGRATION $I_L \to I_H$	$(0,1)$	$\eta_L I_L$
`H`	MIGRATION $I_H \to I_L$	$(1,0)$	$\eta_H I_H$
RL, RH	DEATH	$(0,0)$	γI_L, γI_H
W	NEUTRAL	$(0,0)$	ωR
SL, SH	SAMPLE	$(1,0)$, $(0,1)$	ψI_L, ψI_H

Note TH: a superspreader transmits, the parent remains in I_H, and the new case always enters I_L, so its two production slots sit in *different* demes.

The reference is `si2r_model.qmd`, a document living in a separate project directory (`efficiency/si2r/`), never used by, cited by, or consulted during any milestone of the PhyloPOMP.jl compiler. SI2R was declared in this package's DSL for the first time in the session that produced this check. That document independently hand-derives, from the KLI paper, an eighteen-line compatibility-indicator/saturation table, closed-form Φ_u expressions, and a decay equation

$$\lambda = \alpha_{\mathrm{SL}} + \alpha_{\mathrm{SH}} + \alpha_{\mathrm{RL}}\mathbf{1}_{I_L \le \ell_L} + \alpha_{\mathrm{RH}}\mathbf{1}_{I_H \le \ell_H}.$$

Two of its closed forms are used below: its Eq. 132, giving TL's collapsed `:noop` group as $1 - \binom{\ell_L}{2}/\binom{I_L}{2}$, and its Eq. 135, giving TH's collapsed `:noop` group as $1 - \ell_L/I_L$.

The declaration itself (`src/examples/mgp_si2r.jl`) uses the identical `@mgp`/`@event` DSL as SEIR and MERS. The compiler functions run against it—`enumerate_saturations`, `kli_binomial_ratio`, `full_transitions`, `reduce_event_indicator`, `total_decay`—are the generic M02–M04 and M07 code, written for SEIR and MERS, executed *unmodified*. There is no SI2R-specific branch anywhere in the pipeline.

4.5.2 Mapping to the compiler's output: a structurally new case

TL reproduces Layer 4's situation on a third model. Its production $r = (2,0)$ puts both slots in L, and at full saturation the compiler returns

$$\varphi^{\mathrm{TL}}_{\mathrm{fork}} = \frac{1}{\binom{I_L}{2}},$$

the *unordered*-pair Moran/Kingman rate—independent confirmation, on a model that had no part in Layer 4, of the same identity.

TH is the genuinely new case. Its production $r = (1,1)$ places one slot in L and one in H, so its Fork saturation $s = (1,1)$ spans two *different* demes. No previously declared model in this codebase had a reachable two-slot Fork of that shape: MERS's same-deme forks (TCC, THH) have both slots in one deme, and MERS's cross-deme marks (THC, TCH) are $r = (1,1)$ but only ever realise a single slot—they never reach a Fork at all. The generic code, unprompted, returns

$$\varphi^{\mathrm{TH}}_{\mathrm{fork}} = \frac{\binom{I_L-\ell_L}{0}}{\binom{I_L}{1}} \cdot \frac{\binom{I_H-\ell_H}{0}}{\binom{I_H}{1}} = \frac{1}{I_L\, I_H},$$

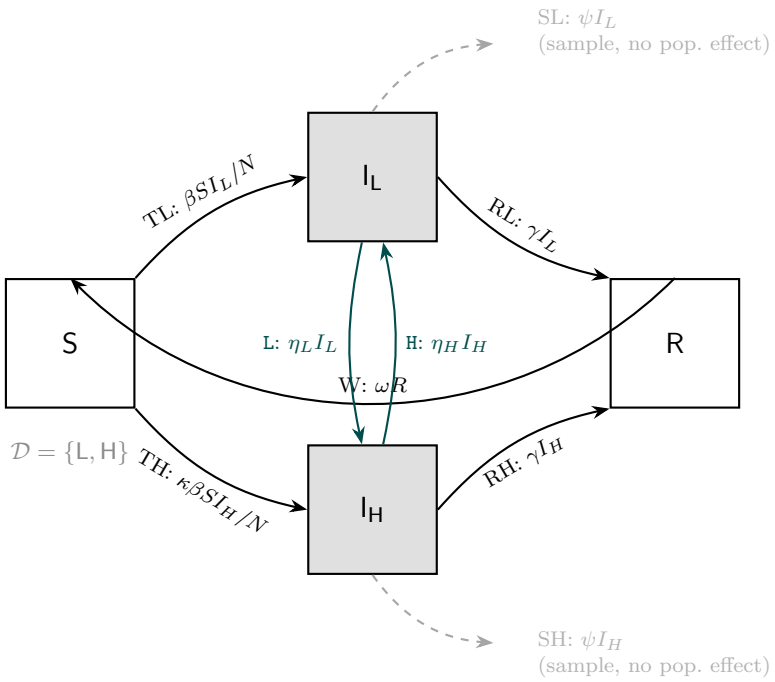


Figure 4: The SI2R (superspreading) population model, redrawn from `si2r_model.qmd` in this report's house style (shaded boxes are the two demes $\mathcal{D} = \{\mathsf{L}, \mathsf{H}\}$, matching `si2r_model.qmd`'s own convention). `TL` and `TH` are BIRTH marks; `L` and `H` (teal) are MIGRATION marks between the two demes; `RL`/`RH` are DEATH; `W` is NEUTRAL; `SL`/`SH` (dashed, gray) are SAMPLE marks with no effect on the population state x — a sample changes the genealogy's coloring but not $\mathbf{X}_t$, so the dashed stub does not connect to another compartment.

an *ordered*-pair rate—one designated slot per deme, no factor of 2 and no binomial—which is exactly the denominator $I'_L I'_H$ appearing in the QMD's boost equation. The unordered/ordered distinction is not encoded anywhere in `kli_binomial_ratio`; it falls out of the production vector. A pipeline that had been pattern-matched to MERS's fork template would not produce it.

4.5.3 Numerical verification

`test/kli_si2r_test.jl` performs 9276 assertions across eleven independent sub-testsets, in exact `Rational{Int}` arithmetic throughout, with no floating-point tolerance anywhere. All 9276 pass. For the four regular BIRTH and MIGRATION marks, the compiler's φ_u was compared saturation-by-saturation against the QMD's binomial-ratio formulas over 100 randomly sampled states, under a non-vacuity guard requiring `enumerate_saturations` to return exactly $\prod_d(\min(r^u_d, \ell_d) + 1)$ entries—without which a short or empty enumeration would let the comparison loop assert nothing:

Event mark	r^u	**# saturations checked**	**Match**
TL (low-rate transmission)	$(2, 0)$	3/3 per state, 100 states	exact
TH (superspreader transmission)	$(1, 1)$	4/4 per state, 100 states	exact
`L` (`I_L` → `I_H`)	$(0, 1)$	2/2 per state, 100 states	exact
`H` (`I_H` → `I_L`)	$(1, 0)$	2/2 per state, 100 states	exact

The remaining sub-testsets check: the structural fields (r^u, Δ_u, hazard closures) of all nine marks against the QMD's α_u column; the `full_transitions` classification (Identity / InlineSameDeme / CrossDeme / Fork) at a concrete state against the QMD's stated operator column ($Q_u = 1$ with $y' = y$ / swap / fork); the Chu–Vandermonde

identity; `total_decay` against the decay equation above, with a non-vacuity guard forcing the $I \leq \ell$ boundary branch actually to fire; TL's fork against $1/\binom{I_L}{2}$; and TH's fork against $1/(I_L I_H)$.

Remark 26 (Weighted versus unweighted collapse)**.** The QMD's claim that its table lines 1&2 and lines 4&5 collapse was checked in two distinct ways, because the obvious reading of it is wrong. As a *grouping-structure* assertion—does `reduce_event_indicator` produce exactly the right number of groups, with the right transition types in each—it holds directly. But the raw group sum $\Phi_u = \sum_s \varphi_u(s)$ of Eq. (4) is *unweighted*, and is a different, generally smaller quantity than the QMD's closed forms; the two coincide only when the multiplicity is one. Those forms are recovered with the multiplicity weight, as $\varphi_{\mathrm{id}} + \binom{\ell_L}{1}\varphi_{\mathrm{inline}} = 1 - \binom{\ell_L}{2}/\binom{I_L}{2}$ for TL (QMD Eq. 132), whose inline slot lies in L, and $\varphi_{\mathrm{id}} + \binom{\ell_H}{1}\varphi_{\mathrm{inline}} = 1 - \ell_L/I_L$ for TH (QMD Eq. 135), whose inline slot lies in H even though the resulting expression is in ℓ_L and I_L. This is the same weighting subtlety found at M09 for MERS's TCC/THH aggregates; it was rediscovered independently here from the disagreement itself, not assumed from the earlier finding. Both readings are asserted separately in the test.

The comparison against Eqs. 132 and 135 is worth separating from the rest. Checking the compiler's φ_u against the QMD's product-form binomial-ratio formulas is, in the end, two implementations of one formula agreeing—useful, but weak evidence. Eqs. 132 and 135 are algebraically *simplified* closed forms: the binomials have cancelled, and the resulting expression no longer resembles the compiler's computation at all. Agreement there is a substantially stronger signal.

4.5.4 The falsification experiment

A test suite that cannot fail proves nothing. To establish that these 9276 assertions actually discriminate a correct declaration from an incorrect one, two deliberate bugs were introduced into `src/examples/mgp_si2r.jl` and the suite re-run. Each was reverted immediately afterwards.

Mutation	**What changed**	**Failed / total**	**Sub-testsets that caught it**
1 — wrong deme in hazard	TH's rate $\kappa\beta SI_H/N \to \kappa\beta SI_L/N$ (copy-paste of TL's I_L)	2 / 9276	1 of 11: the hazard-versus-α_u testset only
2 — same-deme fork	TH's `move=fork(I_H => I_L, I_H)` → `fork(I_H => I_H, I_H)`	827 failures + 7 errors / 8885	6 of 11: production-vector structure, φ_u values, classification, collapse grouping, TH ordered-fork identity, QMD simplified closed forms

Mutation 1 is the plausible copy-paste slip: the wrong deme's population in one hazard. Exactly two assertions failed, both inside the sub-testset that compares hazard closures against the QMD's α_u column. The other 9274—every φ_u, Φ_u, decay, and classification assertion—passed unchanged, because none of that combinatorial machinery reads the hazard closure at all. That is the correct behaviour, not a missed detection: those checks are hazard-blind by construction, and the one testset that is not caught it.

Mutation 2 is the structural bug: it makes TH a same-deme fork, which is precisely what a compiler merely pattern-matching MERS's TCC template onto TH would emit. 827 assertions failed and 7 errored, out of 8885. (The total is lower than 9276 because changing r^u from $(1,1)$ to $(0,2)$ also changes the size of `enumerate_saturations`'s output, so the comparison loops run fewer iterations.) The failures were spread across six of the eleven independent sub-testsets simultaneously—the raw production-vector check, the φ_u values, the transition classification, the collapse grouping, the TH-specific ordered-fork identity, and the QMD's simplified closed forms all disagreed at once.

Two sub-testsets did *not* catch it and passed entirely unchanged: the Chu–Vandermonde identity (800/800) and the Φ_u-conservation check (3244/3244). This is worth stating plainly rather than glossing over, since it delimits what those checks can establish. Both are self-consistency identities: they verify that φ_u sums to one over a mark's saturations, and that each Φ_u equals the sum of its own group members. Those statements hold for *any* well-formed declared model, correct or not, because they test only the compiler's internal bookkeeping and never compare against an external reference. They are evidence that the arithmetic is coherent; they are not evidence that the model is right. The remaining three sub-testsets—hazards, decay, and TL's fork—do not depend on TH's production structure and had no reason to change.

Both mutations were reverted, the model file confirmed byte-identical to its pre-mutation state, and the full 30542-assertion suite re-run green.

4.5.5 Why this genuinely helps

The logical structure differs from Layer 4's and should not be conflated with it. Layer 4 is an *external-reference* check: it compares one compiler output against a textbook result that predates the entire project, at a single fixed structural pattern. Layer 5 is a *generalisation-plus-falsifiability* check, and its reference is external to this project's *development* rather than to its *authorship*—the QMD is a separate document, in a separate project, written without reference to this compiler and never consulted during any milestone, but it is not literature-external in the way the Moran formula is. Layer 5 therefore does not do what Layer 4 does, and does not replace it.

What it does establish has two parts. First, generality: the M02–M04 and M07 functions were written before SI2R existed here, contain no SI2R-specific branch, and nonetheless reproduce an independently hand-derived filter—including a production shape, the cross-deme $r = (1, 1)$ fork, that no model available during their development could exercise. Fitting-to-the-development-models is ruled out for the slice checked, because the new case's answer ($1/(I_L I_H)$, ordered) is numerically different from anything the development models could have taught the code.

Second, and more decisively, falsifiability. The worry that motivates Section 4 is a pipeline that echoes back whatever the DSL declared, so that any declaration "verifies" against any reference. If that were the situation, Mutation 2 would have sailed through all 8885 assertions the way Mutation 1 sailed through 9274 of 9276. It did not: two-thirds of the independent sub-testsets diverged, in exactly the places the underlying combinatorics says a changed production vector must show up. The suite demonstrably distinguishes a correct SI2R declaration from an incorrect one.

4.5.6 Honest scope

Layer 5 validates the generality of the M02–M04 pipeline (φ_u via `kli_binomial_ratio`, `full_transitions`, `reduce_event_indicator`) and of M07's `total_decay` across a genuinely new structural case, against a reference authored outside this project's development—and it demonstrates by direct falsification that the test suite discriminates rather than being vacuously true. It does not cover:

- **Any compiled filter for SI2R.** No `mgp_si2r_filter.jl` and no `si2r_naive.jl` exist. SI2R therefore has no Gate-5-style numerical-equivalence check—nothing comparable to Layer 3—and none of the driver assembly, proposal-kernel, or SMC machinery is exercised for it.
- **The QMD's driver and boost equations as assembled expressions.** Only selected φ_u and Φ_u *values* that those equations quote as intermediate quantities were checked, not the equations themselves.
- **Parameter regimes beyond the states swept.** The φ_u comparison covers 100 sampled states with $I_L, I_H \in [2, 50]$; the classification and collapse checks are made at concrete states. Nothing is claimed for regimes outside those.

4.6 Additional evidence: property-based testing (M12)

Beyond the four layers above, M12 added 14,489 randomized property tests across five structural invariants, all passing:

Property	Statement	Trials
P1	$\ell_d \leq n_d$ preserved by all functions	400
P2	$\sum \varphi_u = \sum \Phi_u$ (exact $\mathbb{Q}$)	500
P3	$\Phi_u \geq 0$ and $\varphi_u \geq 0$	500
P4	Infeasible inputs $\rightarrow$ zero, never error	400
P5	Structural invariants on every transition type	600

4.7 What remains

The honest summary, stated plainly rather than inflated:

- **No model has "exact oracle" status.** The strongest evidence is "handwritten implementation comparison" (Gate 5), for SEIR and MERS only. SI2R has no compiled filter at all, so it has no Gate 5 evidence of any kind—its verification stops at the M02–M04/M07 compiler stages.
- **The Moran check (Layer 4) is the only external check.** It covers the Fork/φ_u slice of the compiler. The structured/multi-deme pieces (driver, decay, cross-deme migration, the m-reduction) remain validated only by internal cross-checks.

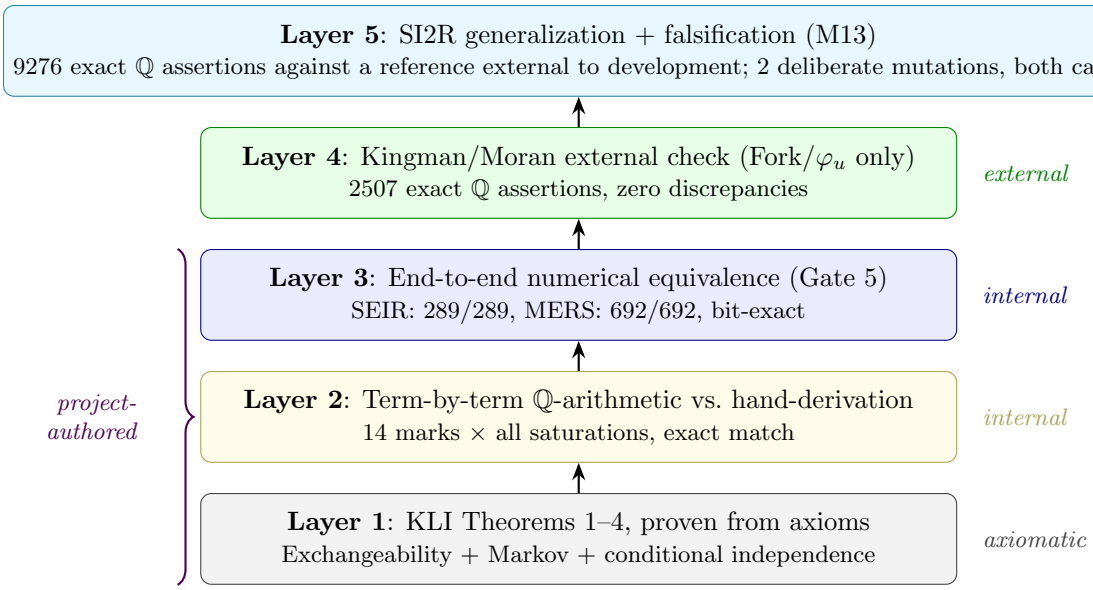


Figure 5: The five layers of verification, in three distinct categories. Layer 4 (green) is *literature-external*: it checks against a textbook combinatorial result (Kingman/Moran) that predates KLI and this project entirely. Layer 5 (cyan) is *development-external*: it checks against SI2R (`si2r_model.qmd`), a model specification never consulted during any milestone of this compiler's development, but not itself a source independent of this research group the way the Moran formula is — a different, weaker-but-still-genuine kind of externality, strengthened here by a direct falsifiability check (deliberate mutation of the model declaration) that Layers 1–4 do not perform. Layers 1–3 (grey, yellow, blue) are internally consistent and mutually reinforcing, but entirely project-authored, with no external or development-external reference at all.

- **SI2R occupies a middle position.** It is no longer "not yet independently verified": its φ_u, transition classification, reduction grouping, and decay were checked term-by-term against a reference document authored outside this project's development (Layer 5), and the check was shown by falsification to be capable of failing. But SI2R still has no `mgp_si2r_filter.jl` and no `si2r_naive.jl`, so it lacks the Gate-5 compiled-versus-hand-coded numerical-equivalence check that SEIR and MERS have. Closing that gap is the obvious next step.
- **BDEI and BDSS** remain "not yet independently verified" in the original sense: neither model exists in the codebase at all.
- The KLI framework's actual novel content—multi-deme genealogies, migration, decay/boost corrections—does not reduce to a textbook special case, so no external check of comparable rigor is available for those pieces. The Kingman/Moran check is as far as the existing literature allows.

Acknowledgments

This document was generated from a review of milestones M00–M13 of the PhyloPOMP.jl compiler project. The mathematical framework is due to King, Lin, and Ionides.